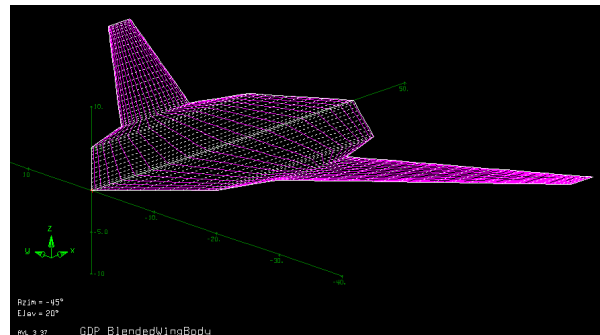
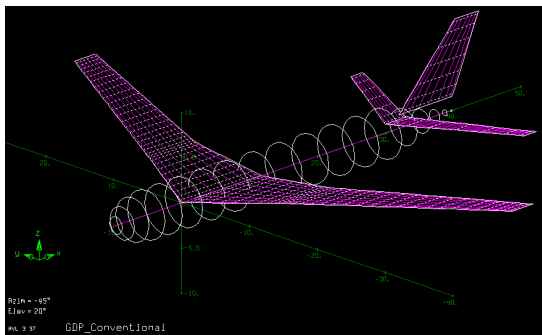


Hydrogen Powered Aircraft

Group Design Project - Individual Report

HPA03: Aerodynamics



Academic supervisor: Dr Chris Cantwell
Department: Department of Aeronautics
Module: AERO96005 Group Design Project
Academic year: 2020/2021
Student: Isaac Morgado Santos
CID: 01499175
Date: June 14, 2021

Department of Aeronautics
South Kensington Campus
Imperial College London
London SW7 2AZ
U.K.

Abstract

This report focuses on the aerodynamic analysis of the wing planforms for two hydrogen aircraft designs: a conventional one and a blended wing body. AVL was used to analyse lift and induced drag, and a rough spanwise XFOIL integration was used to predict viscous drag. Results were compared to Raymer methods and were found to be quite close. Reasoning behind aerostructural optimisation of the conventional wing was provided, the merit function (range) defined and the results, reasoning and constraints shown.

Group Breakdown

The table below details the group breakdown and all group members

FATT Group	Name	CID
FATT 01 Configuration Definition	Puyuan Jin	01515264
	Helen Kirkpatrick	01343388
	Eric Leung	01486521
	Abilan Vijayaratnam	01503705
FATT 02 Propulsion Systems	Shrai Chaddah	01524813
	Ho Yin Ma	01509720
	Zan Jia Tan	01325898
	Pannita Tregalnon	01517832
FATT 03 Aerodynamics	Anudi Bandara	01508055
	Jose Berberoff Naval	01503303
	Isaac Morgado Santos	01499175
	Ho Jhoon Andy Sung	01489872
FATT 04 Structural Design & Manufacturing	Claudia Jimenez Cuesta	01508139
	Bilal Sattar	01501347
	Aishwarya Swaminathan Saravanan	01496622
	Wing Kin Yu	01505340
FATT 05 Flight Dynamics & Control	Ray Asahara Thio	01325502
	Pablo de Felipe Schepers	01502079
	Javier Guerra Vence	01510959
	Jeremy Tian Wen Loong	01482408
FATT 06 Aircraft Ground Infrastructure, Energy Storage & Operation Costs	Nontanasorn Budninpech	01552239
	Jakub Porzycki	01344558
	Lucas Starck	01502397
	Shaun Tan	01386231

Contents

List of Figures	v
List of Tables	v
1 Introduction	1
1.1 Report Objectives	1
2 Aerodynamic Analysis	1
2.1 Planform Definition	1
2.2 VLM Theory	2
2.3 Viscous Drag Prediction	3
2.4 Code and Analysis Procedure	4
3 Results, Validation and Discussion	6
4 Aerostructural Optimisation of Conventional Design	8
4.1 Merit Function	8
4.2 Optimisable Parameters	9
4.3 Aspect ratio optimisation	10
4.4 Taper and twist optimisation	10
5 Conclusion	12
Bibliography	13
Appendices	14
A Additional Plots	14
B Code - Conventional Design	16
B.1 conv_main	16
B.2 conv_multipleAnalyses	18
B.3 conv_buildup	21
B.4 conv_planformcalcs	22
B.5 conv_planformplot	24
B.6 conv_avlin	25
B.7 conv_avlrun	29
B.8 conv_avlout	30
B.9 conv_outproc	31
B.10 conv_chordwisedistrib	34
B.11 conv_controlSurfaceIn	36
C Code - Blended Wing Body	39
C.1 bwb_main	39
C.2 bwb_multipleAnalyses	40
C.3 bwb_buildup	44
C.4 bwb_planformplot	45
C.5 bwb_avlin	46
C.6 bwb_avlrun	49
C.7 bwb_avlout	50
C.8 bwb_outproc	51

C.9	bwb_chordwisedistrib	54
D	Code - XFOIL Analysis	57
D.1	f01_run1re	57
D.2	f02_save1re	59
D.3	f03_plotALLre	60
E	Code - Common Helper Scripts	62
E.1	myAvlFileRead	62
E.2	spacing	62
E.3	structures_chordwise_interp	63

List of Figures

2	Final Planforms.	2
3	VLM panels.	2
4	Blending of aerofoil aerodynamics.	4
5	Stall prediction.	5
6	Spanwise lift distributions. Note C_L here is normalised with MAC and not sectional chord.	6
7	Planforms and equivalent wings.	7
8	Effect of aspect ratio on merit index.	10
9	First taper/twist analysis.	11
10	Second taper/twist analysis.	11
11	Third and final taper/twist analysis.	12
12	Panel detail.	14
13	Stall plots.	14
14	Spanwise and chordwise lift distribution.	15

List of Tables

1	Wing aerodynamic parameters at cruise.	6
2	Validation of cruise lift and drag for just the wing.	7
3	Changes to conventional design. θ_1 and θ_2 are twist angles for mid and tip sections.	10

1 Introduction

The purpose of the project was to design and analyse the feasibility of a hydrogen-powered mid-market commercial aircraft, with design specs given in the brief [1]. Two concepts were designed and analysed - a conventional design and a blended-wing body, with the goal of determining if either (or both) are viable and which is more desirable than the other. By the end, hopefully, an economically viable hydrogen-powered aircraft should be designed.

1.1 Report Objectives

This report deals mainly with the aerodynamic analysis of the wings for both designs. The analysis methods used will be presented, justified and explained; the underlying theory briefly addressed; the code architecture and interaction with other subteams explained; and the process and results of wing aerostructural optimisation will be given.

2 Aerodynamic Analysis

This section concerns the analysis methods used for the wing (with option to also analyse the tail, fuselage, control surfaces, HLDs and winglets) during design of said surfaces. Both wing designs (conventional and blended wing body) feature complex multi-section planforms, which have a large number of parameters to be designed (and possibly optimised for). One could use semi-empirical methods such as those in Raymer [2] to model lift and drag, but these methods fail to model certain parameters (taper, sweep, aerofoils used) and work only with single-section wings. It is therefore useful to seek more precise methods that respond and model changes in every design parameter - this would help the members of the team designing the wings as they could see the effect of changing each parameter on the lift, drag and other aerodynamic outputs.

Upon looking at the different methods available (LLT - lifting line theory, VLM - vortex lattice method, or panel methods), Peerlings [3] gave VLM as the simplest that gives quite reasonable accuracy. A freeware VLM program, AVL (Athena Vortex Lattice, [4]) was used to save coding time, increase accuracy (most freeware VLMs make modifications and corrections to the method which can be quite complex and sometimes undocumented) and ensure the results given are somewhat valid [3].

AVL can model lift, induced drag, and their distributions along the span. However, viscous drag cannot be modelled with a vortex lattice as it is an inviscid method. Raymer methods for drag prediction (coded and used by Sung [5]) are not sensitive to all planform parameters, so an alternative method was used: discretising the wing along the span, analysing each section in XFOIL and integrating. This method neglects 3D effects but is sensitive to changes in all planform parameters, making it ideal for designing.

2.1 Planform Definition

As was discussed by Bandara [6] and Berberoff Naval [7], the conventional design features a wing yehudi and constant leading edge sweep along the span, for both aerodynamic and structural benefit, as well as aiding in positioning and retracting the landing gear. This is effectively a two-section cranked wing, that is simply constrained so that the leading edge sweep is constant for both sections and that the inner-section trailing edge is unswept. The BWB (blended wing body) design is a three-section cranked wing, which was found to be necessary to fit enough fuel tank volume inside the wing and also aid in modifying the aerodynamic centre, a big help when analysing the stability and trim of the aircraft.

The final planforms for both designs are shown in Figure 2. As can be seen, both designs are, in essence, cranked wings, with different number of sections and different constraints on geometry. This is quite convenient as AVL builds a wing by inputting geometrical and aerofoil parameters at different sections along the span, so the code for both designs can be re-used with slight modifications.

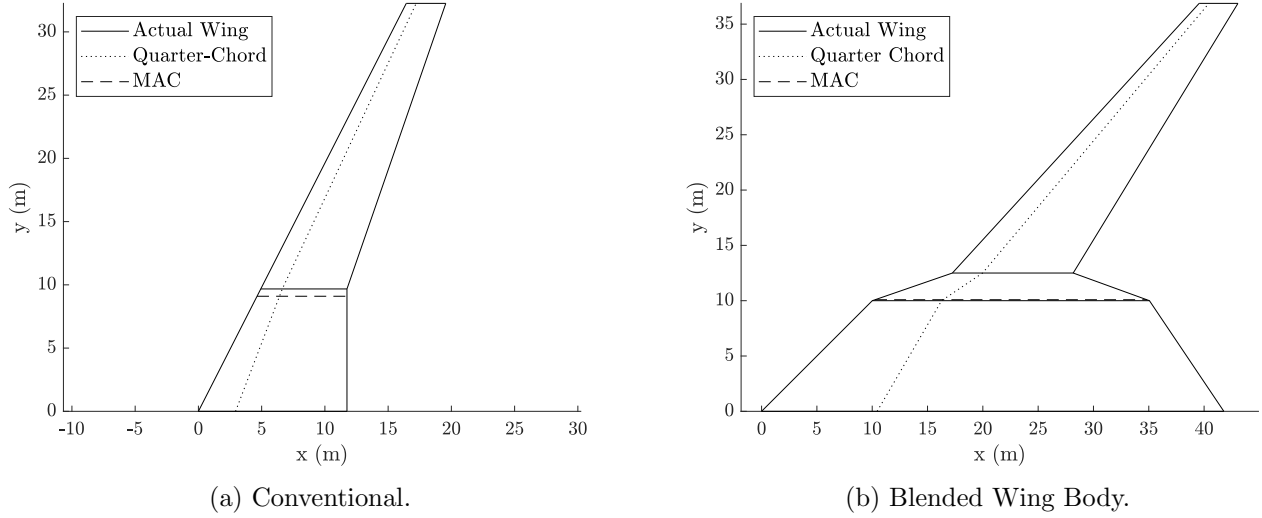


Figure 2: Final Planforms.

In order to define the wing geometry as required by AVL (which usually involves finding leading edge positions, as well as the various chords), equations ESDU 76003 [8] were used. This allowed for full calculation of any wing parameter needed, from the few design parameters such as taper, sweep, etc.

2.2 VLM Theory

The classical Vortex Lattice theory has been explained extensively in literature [9]. The full equations used are not worth providing here, but a general understanding of the method is.

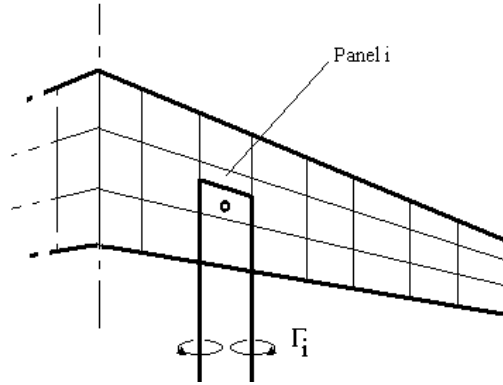


Figure 3: VLM panels.

The wing is discretised into a number of flat panels, both in the chordwise and spanwise directions (shown in Figure 3). A horseshoe vortex is placed on each panel, with trailing legs extending downstream and a bound vortex placed on the panel's $c/4$ line. A control point is placed on the mid-span of the panel's $3c/4$ line. Using Biot-Savart's law, the velocity $\mathbf{v}$ induced by every j^{th} panel on each i^{th} panel can be found:

$$\mathbf{v}_i = \sum_{j=1}^N \mathbf{w}_{ij} \Gamma_j \quad (1)$$

Where $\mathbf{w}_{ij}$ is quite a complex expression generated from Biot-Savart (a function of the panel geometry and control points only). Applying an impermeability condition at each control point yields:

$$\left(\mathbf{V}_\infty + \sum_{j=1}^N \mathbf{w}_{ij} \Gamma_j \right) \cdot \mathbf{n}_i = 0 \quad (2)$$

Where $\mathbf{n}_i$ is the unit vector normal to the camber line at the panel's control point. Rearranging:

$$a_{ij} \Gamma_j = b_i \quad (3)$$

Which is just a system of equations, where:

$$a_{ij} = \mathbf{w}_{ij} \cdot \mathbf{n}_i \quad ; \quad b_i = V_\infty \begin{pmatrix} \cos \alpha \cos \beta \\ -\sin \beta \\ \sin \alpha \cos \beta \end{pmatrix} \cdot \mathbf{n}_i \quad (4)$$

This system can be solved for the vortex strengths Γ_j , which can then be used to calculate induced velocities and forces generated by each panel (lift and induced drag) using the Kutta-Joukowski theorem.

While this method is relatively simple to perform computationally and widely-used, its limitations must be addressed. Firstly, the assumption of thin aerofoils at small angles of attack. Small α is not fully valid for stall prediction (conventional wing stalls at 27°). Assuming thin aerofoils is somewhat accurate for the conventional wing, but the BWB root t/c of 16% is more problematic. AVL includes a correction for the aerofoil thickness which, although optimistic, was used for the root of the BWB to improve accuracy.

VLM is also an incompressible and inviscid method. Viscous drag cannot be predicted, and thus an alternative method presented in the next section was used. At the cruise Mach number of 0.8, flow is by no means compressible. AVL uses Kármán-Tsien to correct for compressibility, which is non-ideal but good enough for this stage of design.

Finally, it is worth noting that AVL is an extended Vortex Lattice Method, allowing for modelling of slender bodies (fuselage), as well as trailing edge devices (flaps and ailerons). Functionality for modelling these components was added to the code, as it would prove to be very useful for other subteam's analysis (especially HPA05 - Flight Dynamics and Control).

2.3 Viscous Drag Prediction

As previously described, a 2D analysis and spanwise integration method was used to estimate the drag of the wing, for design iteration and optimisation. The initial plan was to divide the wing into many sections spanwise. For each section, the absolute angle of attack could be found by summing wing incidence, sectional twist, overall angle of attack and sectional induced angle of attack (from AVL):

$$\alpha_{sect} = i_W + \theta_{sect} + \alpha - \alpha_{i_{sect}} \quad (5)$$

Knowing the aerofoil coordinates, as well as the Re of each spanwise section, XFOIL [10] could be run at this AoA to give a crude estimate of the sectional C_d . This could then be integrated along the span to give the wing viscous C_D . For wing sections that blended between two aerofoils, the “in-between” aerofoil coordinates could be calculated and XFOIL run as described above.

However, XFOIL is prone to crashing, freezing or failing to converge at certain Re/α combinations, particularly with supercritical aerofoils. This would prevent the code from running to completion or fail to provide results. To solve this, a cruder method was used. Before analysing the wing, XFOIL would have to be manually run for each aerofoil, at select Re and α values. The resulting C_d and corresponding Re and α were stored in a “lookup table”. Then, during execution of the wing analysis script, the code could simply

use the lookup table to interpolate the C_d for the desired Re and α . For spanwise sections where geometry would blend between two aerofoils, a weighted average of the two aerofoils' C_d was used:

$$C_d = (1 + \phi) C_{d,1} + \phi C_{d,2} \quad (6)$$

Where ϕ is a blend fraction going from 0 (aerofoil 1) to 1 (aerofoil 2). A visual representation is shown in Figure 4 (note the root and mid (30% span) aerofoils are the same). As can be seen, The blended C_D distribution matches the root/mid XFOIL data at the start. The mid and tip sections have different aerofoils, so we model a linear transition between the two curves.

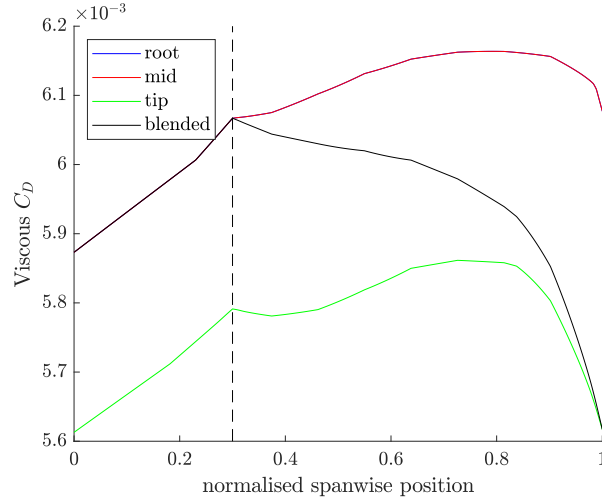


Figure 4: Blending of aerofoil aerodynamics.

While less accurate, this method was very computationally cheap at runtime and, most importantly, robust. It simply required some manual labour if the wing aerofoils were to be changed. With regards to accuracy, as explained earlier, 3D drag effects are neglected, and instead of analysing blended aerofoil geometries, the aerodynamic results of the two aerofoils were linearly blended, which is less accurate. Additionally, XFOIL has its limitations in predicting drag. Boundary layer transition is assumed to be instant, and separation is quite poorly modelled, which is especially important when analysing the BWB's root sections. However, more accurate methods (such as VGK) either could not be ran within a MATLAB script or required additional inputs such as transition x/c that we could not easily predict. CFD was too complex for the scope of this project, and not suited at all to conceptual design stages. So XFOIL was used, despite its slight inaccuracies.

2.4 Code and Analysis Procedure

Interfacing with AVL requires generating input text files, running AVL through the command line, generating output text files and post-processing in MATLAB. The code architecture is as follows:

1. Define and assign all the input parameters. These include all the values needed to fully constrain the planform geometry, the Mach and C_L required for the analysis, number of panels to be used, etc.
2. Using ESDU equations, compute the full wing geometry (exact chords, leading edge positions, etc.).
3. Generate `filename.avl`, an AVL input file containing the wing geometry. At each crank, root and tip, provide geometry and aerofoil coordinates.
4. Generate `filename.run`, an AVL input file with a series of analysis commands and output requests.
5. Run AVL through the command line, within MATLAB.

6. Read the output text files (`filename.ft`, containing outputs for the whole wing, and `filename.fs`, containing parameters that vary along the span) and extract results.
7. As AVL only provides C_L and C_{D_i} , estimate C_{D_v} via XFOIL lookup tables.
8. Process the outputs and generate plots. Print out any values required for design.

The above procedure is sufficient for analysing the wing at cruise conditions and outputting cruise α , wing lift, drag and spanwise lifting distribution. To find wing C_{L_α} and α_0 , the analysis was run at two different angles of attack, and the parameters calculated using:

$$C_{L_\alpha} = \frac{C_{L,2} - C_{L,1}}{\alpha_2 - \alpha_1} \quad ; \quad \alpha_0 = \alpha_1 - \frac{C_{L,1}}{C_{L_\alpha}} \quad (7)$$

To predict the stall of the wing, the spanwise $C_{L_{max}}$ was found (by linearly interpolating aerofoil $C_{L_{max}}$ from XFOIL, similarly to how viscous drag was predicted) Then, a numerical bisection method was used to run the 8 steps outlined above, until only one spanwise section had reached its $C_{L_{max}}$ (see Figure 5 and Appendix A). The stall angle of attack, spanwise position and wing $C_{L_{max}}$ can then be noted down.

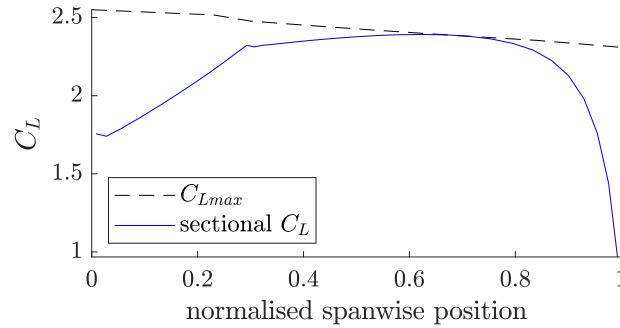


Figure 5: Stall prediction.

Additionally, to aid in the work of the structures and control sub-teams, several modifications to the code were made. Structures required the distribution of force along the whole wing area, not just along the span, so a panel dC_P output was also requested from AVL. This was then multiplied by the panel area to give the force distribution along the wing (plotted in Appendix A).

Controls required modelling of trailing-edge devices (flaps and ailerons), which might or might not start at the already-defined wing spanwise sections. Code was written to generate more wing sections (corresponding to the start/end of these devices), linearly interpolating aerofoil coordinates if needed, and add those sections to the AVL input files. To also use AVL to output stability derivatives, the whole conventional aircraft (including the tail and fuselage) should be modelled. Wing code was recycled for the horizontal and vertical stabilisers, and functionality to also model the fuselage was added.

Finally, the definition of panels used for the vortex lattice analysis should be explained. According to guidelines given by the creator of AVL, a cosine distribution of panels should be used in the chordwise direction. Along the span, panels should be bunched more closely where wing parameters change, so initially a cosine distribution of panels was set for each section of the wing. However, this often led to the spanwise width of panels changing quite drastically between sections. This led to “spikes” in the spanwise lift distributions (see Figure 5, at 30% span). To combat this, the number of panels in each wing section was modified and the distribution was shifted slightly away from cosine and towards equidistant. A total of 640 panels (20 chordwise and 32 spanwise) were used for the conventional wing, and 780 panels (20 chordwise and 39 spanwise) were used for the BWB - the exact geometry can be found in Appendix A.

The main code scripts used for analysis can be found in Appendices B-E.

3 Results, Validation and Discussion

Table 1 contains the results of the aerodynamic analysis of the wing at cruise conditions (steady level flight). $C_{D,v}$ is the viscous drag coefficient, subscript s denotes conditions at stall and y/s is simply the normalised spanwise position (0 at root, 1 at tip). Note that these results are only for the wing and as a result the L/D values given are not representative of the whole aircraft.

Table 1: Wing aerodynamic parameters at cruise.

	C_L	α (°)	$C_{D,v}$	$C_{D,i}$	C_D	C_L/C_D	e	α_s (°)	$(y/s)_s$ (%)
Conventional	0.376	0.730	0.00596	0.00479	0.0107	35.0	0.909	26.6	67.2
Blended Wing Body	0.164	2.58	0.00474	0.00179	0.00653	25.1	0.975	23.1	85.8

The conventional results shown seem perfectly normal: cruise angle of attack is small, viscous and induced drag are roughly equal (showing we are close to the maximum L/D of the wing). Stall AoA seems a bit high but expected due to our rough C_{Lmax} estimation method (and linear AVL analysis) and the position of first stall is inboard of the ailerons, ensuring good stall behaviour. Oswalds will be discussed later.

BWB results also make sense qualitatively. C_L is much lower than that of conventional due to the increase in reference area, and C_D is similarly lower for the same reason. However, viscous drag now contributes roughly 75% to the overall drag. This is expected when considering the large thickness of aerofoils used and large chords relative to span. L/D is lower (as expected, as BWBs sacrifice some aerodynamics to gain benefit in payload capacity and interference drag). Stall angle of attack is lower and happens more outboard, which might lead to tip stall problems. This is somewhat of a necessary evil, however, given the large spanwise length of the elevons (necessary to trim the BWB).

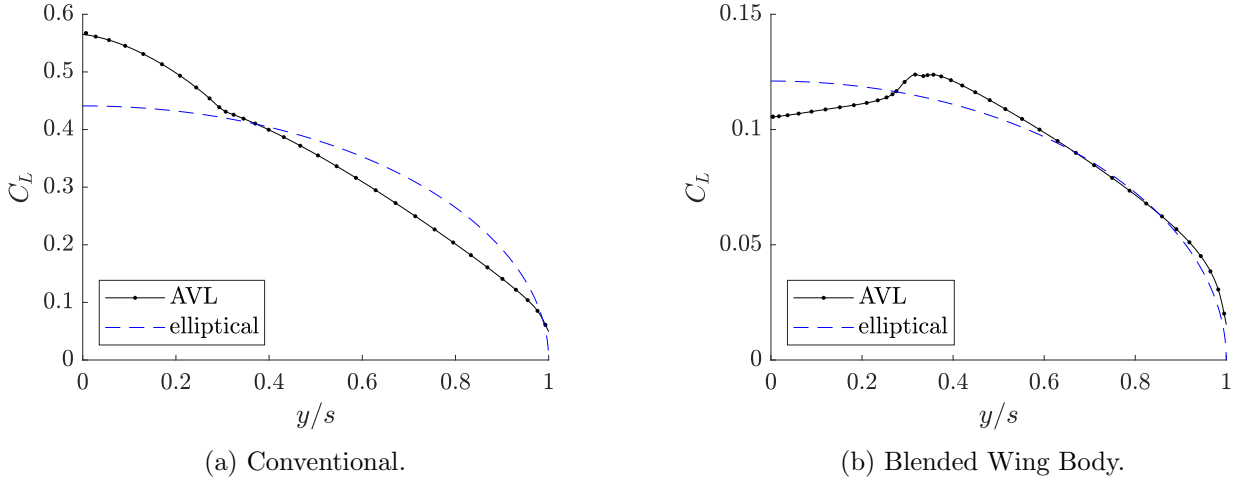


Figure 6: Spanwise lift distributions. Note C_L here is normalised with MAC and not sectional chord.

Spanwise lift distributions for both planforms are shown in Figure 6. There are several interesting things to note. First, the conventional design strays further from elliptical than the BWB (which matches the Oswald values in Table 1). As will be detailed in the next section, the conventional planform went through an aerostructural optimisation, which sacrificed some aerodynamics for structural benefit (and good stall characteristics). The BWB did not for simplicity and due to the complex aerodynamic and stability coupling. As a result, a lower oswalds for the conventional design makes sense, as does the higher lift at the root and lower near the tip. The BWB distribution, on the other hand, shows low lift at the root (due to the inefficient reflex aerofoil used), a spike in lift as the aerofoil transitions to supercritical and still has high chord, and is close to elliptical at the tip (which matches with the high spanwise position of first stall shown in Table 1).

Finally, the accuracy of results must be assessed. Initially, we sought to validate results with other VLM or panel methods. After feedback from the supervisors this was abandoned, as validating two methods that use the same underlying theory with each other is not very useful. Instead, the lift and drag characteristics were compared with the semi-empirical methods found in Raymer, as shown in Table 2. These methods have been explained in more detail by Bandara [6] and Sung [5].

Table 2: Validation of cruise lift and drag for just the wing.

		$C_{L\alpha}$ (rad ⁻¹)	C_{Lmax}	α_0 (°)	$C_{D,v}$	$C_{D,i}$	C_D
Conventional	AVL	4.72	2.33	-5.23	0.00596	0.00479	0.0107
	Raymer	6.32	2.03	-3.80	0.00598	0.00464	0.0106
	% Error	25	13	27	0.45	3.0	1.1
Blended Wing Body	AVL	3.11	1.20	-1.52	0.00474	0.00179	0.00653
	Raymer, full	4.07	1.31	-0.590	0.00476	0.00179	0.00655
	Raymer, outer	2.78	1.47	-1.69	0.00508	0.00179	0.00687
	% Error, full	24	8.8	61	0.46	0.11	0.31
	% Error, outer	11	18.1	10	6.8	0.11	4.9

Note that the Raymer methods used only apply to a single-section wing. As a result, equivalent wings for both planforms were generated using ESDU [8], as can be seen in Figure 7. They also assume one aerofoil for the whole wing - this was taken to be the aerofoil at the location of the MAC and its aerodynamic and geometric properties were used to represent those of the full span of the equivalent wings.

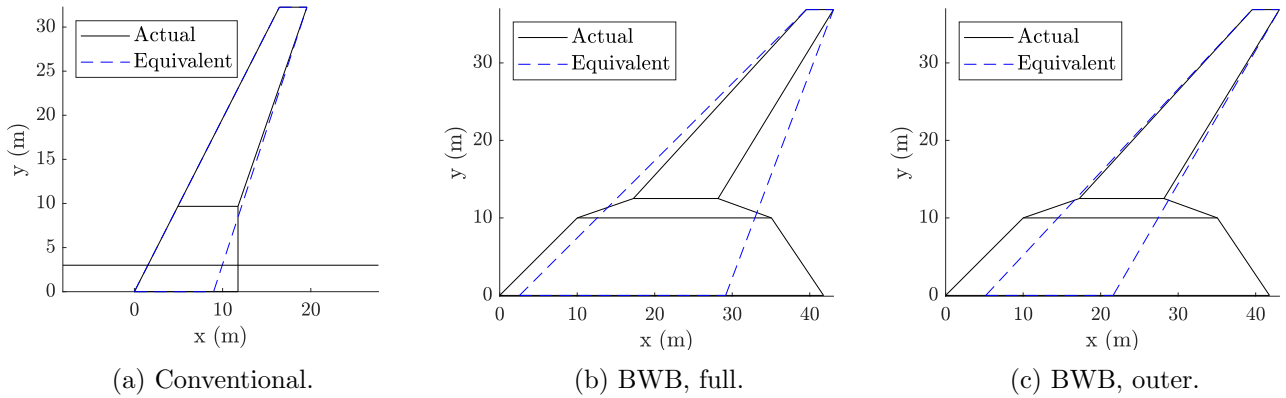


Figure 7: Planforms and equivalent wings.

Drag values match very well, with percentage error of at most a few percent. Lift characteristics, however vary by quite a lot. This is to be expected given the semi-empirical nature of the Raymer methods, when contrasted with the linear lifting analysis provided by AVL. It is nonetheless important to analyse the values.

For the conventional design, lift slope was significantly lower than Raymer’s. This may be due to the assumed aerofoil efficiency factor of 0.95 or an over-prediction of fuselage lift (generated from overspill of the wing lift). C_{Lmax} is slightly above Raymer’s (expected as non-linear effects are not modelled in AVL), and α_0 is considerably more negative. It should be noted that Raymer methods simply take the α_0 of the wing to be that of its aerofoil, which is quite crude, so that may be why the values are so different.

For the BWB design, comparisons are more tricky. The values labelled “full” come from an equivalent wing sized from the full BWB planform (Figure 7b), with its full area exposed. Similar trends to those seen in the conventional values are seen, with AVL underpredicting lift slope, roughly matching in C_{Lmax} and underpredicting α_0 . However, one could argue that the nature of the BWB does not lend itself to comparison with Raymer methods, as root aerofoils are quite thick, and those methods apply to conventional wings only.

If instead, an equivalent wing is sized using only the wing middle and outer sections (Figure 7c), treating the centerbody as a “fuselage” of sorts that overlaps the wing at that section, the lift slope matches much better (note the exposed wing area is now only the area of the middle and outer sections). All other parameters are even more off, as the MAC has changed if we consider the wing to only comprise of the middle and outer sections. Even though the “outer” equivalent wing is much closer to a conventional design, making Raymer more applicable, including the centerbody in the calculation (“full” equivalent wing) gave much better agreement to AVL.

To summarise, drag values matched well. Slope values didn’t match well, C_{Lmax} were reasonably close, and α_0 were very different, but expected given the crudeness of the Raymer method for that parameter. It seems only lift curve slope is the problematic value, and looking into the Raymer equation for it might reveal why:

$$C_{L\alpha} = \frac{2\pi AR}{2 + \sqrt{4 + \frac{R^2\beta^2}{\eta^2} \left(1 + \frac{\tan^2 \Lambda_{max}}{\beta^2}\right)}} \left(\frac{S_{exposed}}{S_{ref}}\right) F \quad (8)$$

Where:

$$\beta = \sqrt{1 - M^2} \quad ; \quad \eta = \frac{C_{l\alpha, aerofoil, M}}{2\pi/\beta} \approx 0.95 \quad ; \quad F = 1.07(1 + d/b)^2 \quad (9)$$

Note d is the fuselage diameter, and η is assumed to be 0.95 due to lack of wind-tunnel data.

It seems that $S_{exposed}/S_{ref}$ and F play quite a large role in the equation. The conventional slope was higher than AVL and had $F > 1$ calculated with the above equation. The “BWB, outer” value was lower and used $F = 1$ (the above formula could not be used as the centerbody cannot be modelled as a cylinder with a specific diameter). The “BWB, full” value was higher and used both $F = 1$ and $S_{exposed}/S_{ref} = 1$ as no part of the aircraft was modelled as a “fuselage”. These results show the large effect $S_{exposed}/S_{ref}$ and F have on values of slope. Without a good way of modelling fuselage overspill, and considering the rough applicability of the method to a blended wing body, exact values cannot be expected to match fully, but can nonetheless be used to “sanity check” the AVL values.

4 Aerostructural Optimisation of Conventional Design

A first iteration of wing design was done by optimising the aerodynamics and designing a structure to go with it. This, however, ignores the multidisciplinary nature of aircraft design: there may be benefits to sacrificing some aerodynamic efficiency if enough weight is saved. In order to quantify this compromise between drag and weight, a suitable merit function (to be maximised) must be derived. Furthermore, the variables to be optimised must be chosen - due to time constraints and limitations in modelling, not all variables were optimised.

Note that this optimisation was not performed for the BWB due to time constraints and the fact that changing the wing planform would heavily affect the stability and trim, which would make the optimisation process far more complex and time-consuming.

4.1 Merit Function

Kenway and Martins [11] suggest that the Breguet Range Equation be used as the merit function to maximise, finding it to be equivalent to minimising TOGW. The paper suggests that in reality, a compromise is made between optimising for TOGW (manufacturing cost + direct operating cost) and for fuel burn (most of the direct operating cost), but for simplicity (and also due to our limited fuel and weight modelling) we have optimised only for TOGW/Range. With W_1 and W_2 being the aircraft weights at start and end of cruise, the equation to be maximised is:

$$R = \frac{V}{TSFC} \frac{C_L}{C_D} \ln \frac{W_1}{W_2} \quad (10)$$

Ignoring velocity (constant) and changes in propulsion (constant TSFC, even though the wing drag, and as a result overall aircraft drag, change), the merit index to be maximised becomes:

$$MI = \frac{C_L}{C_D} \ln \frac{W_1}{W_2} \quad (11)$$

The lift coefficient can be calculated directly from the weight of the aircraft before aerostructural optimisation (ignoring the fuel burn up to the start of cruise, which would simply scale the merit indices equally up/down):

$$C_L = \frac{2 W_{0,start} g}{\rho M^2 \gamma R T S_{ref}} \quad (12)$$

The drag coefficient is simply adjusted by adding in the change in wing drag from the initial starting point to the current iteration (note the initial iteration Cd came from the Raymer component drag estimation [2], performed by Sung [5]), but the wing drag added/subtracted to it comes from the approximate method presented in this report. This is because the component buildup method does not account for changes in certain planform parameters):

$$C_D = C_{D,start} + C_{D,w,current} - C_{D,w,start} \quad (13)$$

And the weights W_1 , W_2 can be similarly adjusted:

$$W_i = W_{i,start} + W_{w,current} - W_{w,start} \quad (14)$$

Where $W_{i,start}$ are the initial sizing weight values for start and end of cruise (assuming W_0 and weight fractions remain constant throughout the optimisation). The wing weights for start and current iteration are estimated from the volume of the structural components sized by Swaminathan Saravanan [12], scaled by a constant factor (which accounts for the added mass of components that have not been designed such as rivets, fasteners, etc.).

Note that in all of this analysis, the “loop” has never been closed, i.e. the aircraft W_0 , design point, etc. have not been updated to reflect changes in wing $\mathcal{R}$, weight, etc. While this is not ideal, it is necessary for fast iteration and shown to be a reasonable assumption [11]. Given the quite minor changes to planform parameters given in following sections, this was a good assumption to make.

4.2 Optimisable Parameters

A full aerostructural optimisation of all planform parameters would have been excessive and not feasible in the time given. Furthermore, some parameters do not make sense to optimise, such as dihedral, sweep and aerofoils used, as these are not chosen for aerodynamic benefit, but rather for other reasons (stability, mach divergence, etc.). As a result, only the aspect ratio, taper and twist (at both mid and tip sections) were optimised. $\mathcal{R}$ is perhaps the most obvious parameter to choose, as aerodynamics and structures want opposite extremes of the value. Taper and twist were also chosen as they are often designed to give an elliptical distribution (with twist also being set to prevent tip stall) but also affect the wing weight by shifting lift towards the root/tip, changing bending moments and thus affecting wing weight.

Optimisation was done by manual iteration (by myself, Bandara [6] and Swaminathan Saravanan [12]). This was simpler and led to critical observation and analysis of results throughout all stages of the optimisation,

as opposed to simply running every possible combination of parameters in code and selecting the most efficient design. As will be seen, rarely were “optimal” values chosen, due to reasoning and constraints that were only developed by this “hands-on” approach.

Initial and final values for these parameters are summarised in Table 3.

Table 3: Changes to conventional design. θ_1 and θ_2 are twist angles for mid and tip sections.

	$\mathcal{AR}$	λ	θ_1 ($^\circ$)	θ_2 ($^\circ$)
Initial	9.51	0.300	-1	-3
Final	10.3	0.266	0	-4

4.3 Aspect ratio optimisation

Aspect ratio was optimised first - it is the most influential design parameter (even leading to changes in the HPA01 design point) and its effect on aerodynamics and structures is quite easy to understand - high $\mathcal{AR}$ will decrease induced drag, but will increase structural weight, so sanity checking results would be quite easy. If results did not match with our expectations, we would not need to re-optimize any other parameters, and could quickly update the formulas/methodologies used.

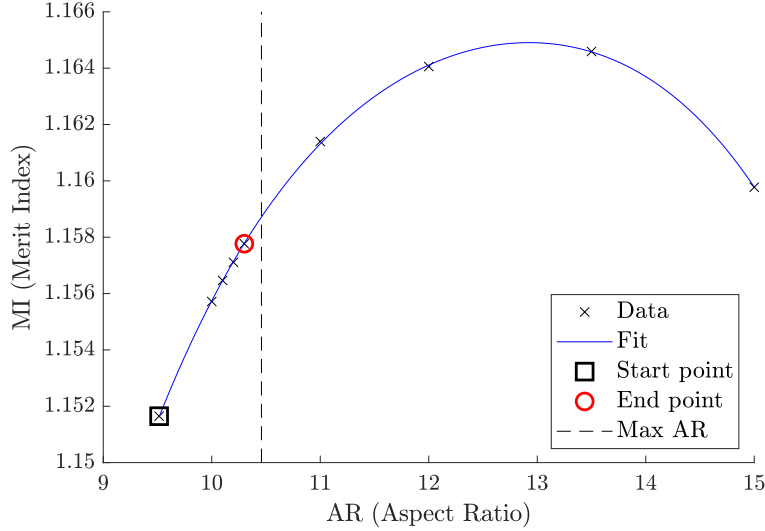


Figure 8: Effect of aspect ratio on merit index.

Figure 8 shows the results of said optimisation. As expected, the merit index starts and ends small, reaching a maximum at around 12.9. However, such a high $\mathcal{AR}$ would lead to a wingspan above 65 m, restricting the aircraft to airports of ICAO code F, which would be very commercially limiting, especially for an aircraft which uses a novel fuel. As a result, an aspect ratio of 10.3 was chosen as it gave a wingspan of 64.5 m, putting the aircraft within ICAO code E.

As can be seen, the aspect ratio increased by about 0.8 - as both points are to the left of the maximum, the drag benefit gained outweighs the structural mass penalty.

4.4 Taper and twist optimisation

The procedure for optimisation of these parameters went was changed thrice while optimising, due to short-sightedness in analysis and failure to consider external constraints on the design.

Initially, twist was kept constant and taper varied, leading to an optimal taper ratio of below 0.2. However, as can be seen in Figure 9b, such an optimum would result in quite large tip stall.

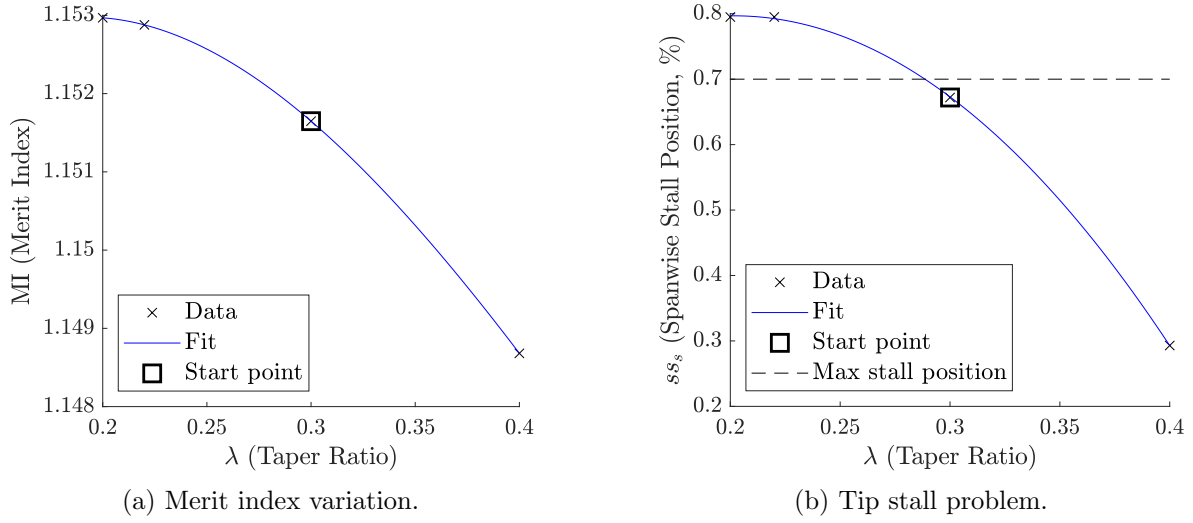


Figure 9: First taper/twist analysis.

To combat this a new analysis was performed. For each value of taper, the tip twist was adjusted to stall the wing at precisely 70% of the span (where ailerons start). Twist at the mid-section was set to zero (as will be seen later, increasing twist here will worsen tip stall). The results are shown in Figure 10, showing a clear optimum at around 0.25. However, the tip twist required to prevent tip stall would be -5.3 degrees, which is quite high. To prevent poor off-design performance, Sadraey [13] proposed a maximum value of -4 degrees for twist at the tip, corresponding to a taper ratio of 0.266. This was thus chosen as the final taper value, as it was the closest to the optimum while still giving reasonable off-design performance. This is lower than the starting value, showing some aerodynamics has been sacrificed for lower structural weight.

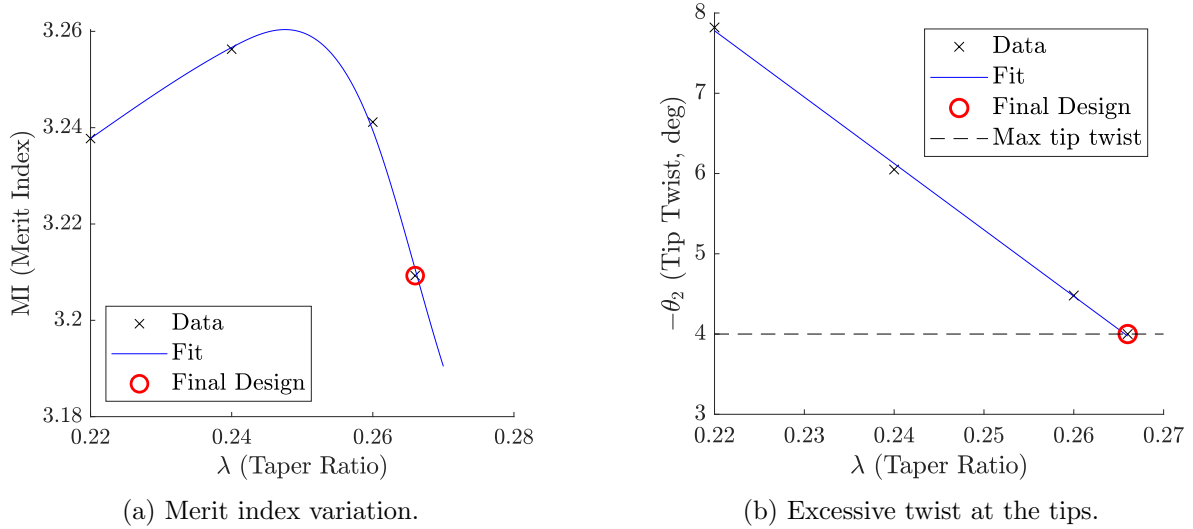


Figure 10: Second taper/twist analysis.

Finally, a twist for the mid-section must be chosen. The previous analysis assumed 0 degrees, but how would the analysis change if this twist was changed? Well, following with the logic provided by the previous two analyses, it is desirable to fix the position of first stall at 70% of the span and to set the tip twist to -4 degrees. With these in mind, for each taper ratio value being analysed, the twist at the mid-section should be changed to achieve stall at the desired spanwise position. The results are shown in Figure 11, showing that an untwisted midsection, corresponding to the previously calculated taper of 0.266, is optimal. The more twisted the mid-section is, the higher the angle of attack of the wing to produce the required lift, but

also the higher the twist at 70% span. The figure shows that the AoA cost outweighs the twist benefit.

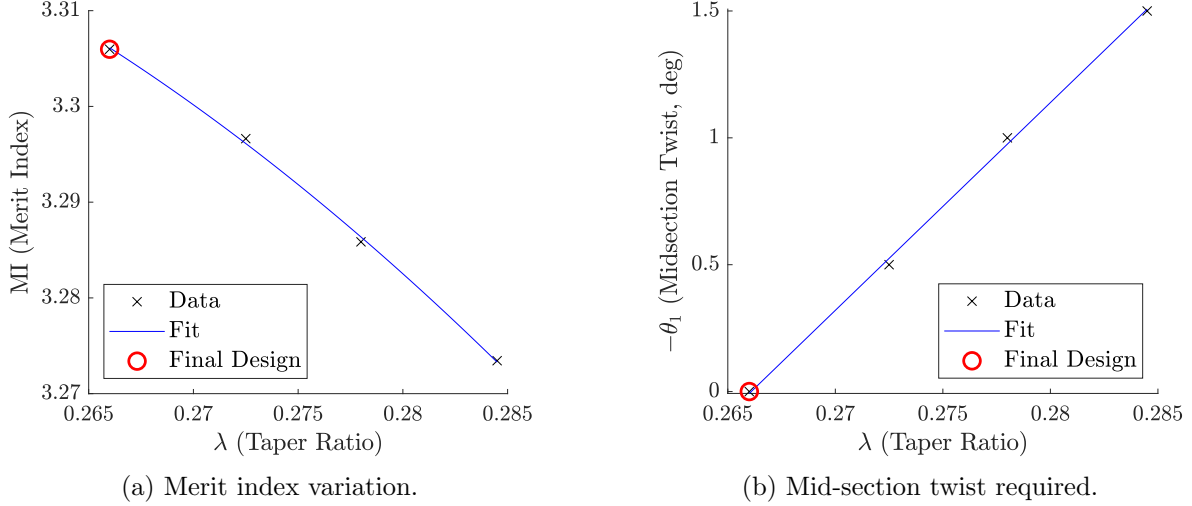


Figure 11: Third and final taper/twist analysis.

5 Conclusion

Given the rough nature of Raymer aerodynamic analysis methods, and in order to aid in the design process, a commercial VLM script (AVL) was used. Code was written to interface with it, feeding it inputs and processing its outputs. Modifications were added to the code to suit the needs of other sub-teams, and throughout the project several versions of the code were being used by several different people to achieve very different results (design of planforms, input to structural analysis, design of HLDs, analysis of stability and trim, etc.).

A crude viscous drag estimation method was also developed for the wing, to ensure drag values changed if any wing design parameter was changed (which would be helpful as designers could see the effect of changing each parameter). XFOIL was used for each section along the span and the integrated to give overall C_{D_v} .

Results from this code were compared to Raymer methods, showing very good agreement in drag values, and errors in the various lifting parameters of up to 30%. Given the rough nature of Raymer methods and their questionable applicability to BWB planforms, these numbers were quite good, and showed agreement between the methods. Sources of error of the values were identified and explained. All other analysis parameters made sense qualitatively and matched well with the design intentions for the two designs.

Finally, the thought process and results behind aerostructural optimisation of the conventional wing was explained. While it is quite hard to validate these results, qualitative analysis of the plots generated were made, which often showed an optimal point as wing design parameters were changed. These optimal points were rarely achievable due to constraints such as tip stall, maximum wingspan and maximum wing twist, but nonetheless values that maximised the merit index were found. In retrospect, the manual iteration procedure used in this optimisation was quite a good idea, as it pushed us towards critically thinking about our results and realising what was achievable or not - a code with nested for loops would have failed to do this and would not have revealed as much to us as using this “hands-on” approach.

If more time had been available, a lot more could have been done. CFD or other more accurate aerodynamic prediction methods could have been used for validation of AVL results. Acquiring wind-tunnel data for the aerofoils at cruise Mach would have been helpful. An aero+structure+stability+trim optimisation of the BWB could have been performed, if only for a couple of parameters for the outer section of the planform. However, the current level of analysis achieved was satisfactory, and the experience of interfacing and working with several people from different subteams on a variety of goals was eye-opening to say the least.

Bibliography

- [1] Hewson R, Palacios R. *GDP: Hydrogen Powered Mid-Market Aircraft*. Department of Aeronautics, Imperial College London; 2021. Available from: https://bb.imperial.ac.uk/bbcswebdav/pid-2103688-dt-content-rid-8675492_1/xid-8675492_1.
- [2] Raymer DP. *Aircraft design: a conceptual approach*. 4th ed. AIAA education series. Reston, Va: American Institute of Aeronautics and Astronautics; 2006.
- [3] Peerlings B. *A review of aerodynamic flow models, solution methods and solvers – and their applicability to aircraft conceptual design*. Delft University of Technology; 2018.
- [4] Drela M, Youngren H. *AVL*; 2017. Available from: <http://web.mit.edu/drela/Public/web/avl/>.
- [5] Sung A. *Hydrogen Powered Aircraft. Group Design Project - Individual Report. HPA03: Aerodynamics*. Imperial College London; 2021.
- [6] Bandara A. *Hydrogen Powered Aircraft. Group Design Project - Individual Report. HPA03: Aerodynamics*. Imperial College London; 2021.
- [7] Berberoff Naval JN. *Hydrogen Powered Aircraft. Group Design Project - Individual Report. HPA03: Aerodynamics*. Imperial College London; 2021.
- [8] Engineering Sciences Data Unit. *ESDU 76003: Geometrical properties of cranked and straight-tapered wing planforms*. Engineering Sciences Data Unit; 1976. (Amendment C made on May 2012).
- [9] Bertin JJ. *Aerodynamics for engineers*. Sixth edition ed. Always Learning. Boston, Massachusetts: Pearson; 2014 - 2014.
- [10] Drela M. *XFOIL*; 2013. Available from: <https://web.mit.edu/drela/Public/web/xfoil/>.
- [11] Kenway GKW, Martins JRRA. *Multipoint High-Fidelity Aerostructural Optimization of a Transport Aircraft Configuration*. Journal of aircraft. 2014;51(1):144–160.
- [12] Swaminathan Saravanan A. *Hydrogen Powered Aircraft. Group Design Project - Individual Report. HPA04: Structural Design & Manufacturing*. Imperial College London; 2021.
- [13] Sadraey MH. *Aircraft design a systems engineering approach* . Aerospace series. Chichester: John Wiley & Sons Ltd; 2012.

Appendices

A Additional Plots

This section contains some additional plots of both wing designs that weren't relevant enough to include in the main report.

Figure 12 shows the chordwise and spanwise panels used in the AVL analysis.

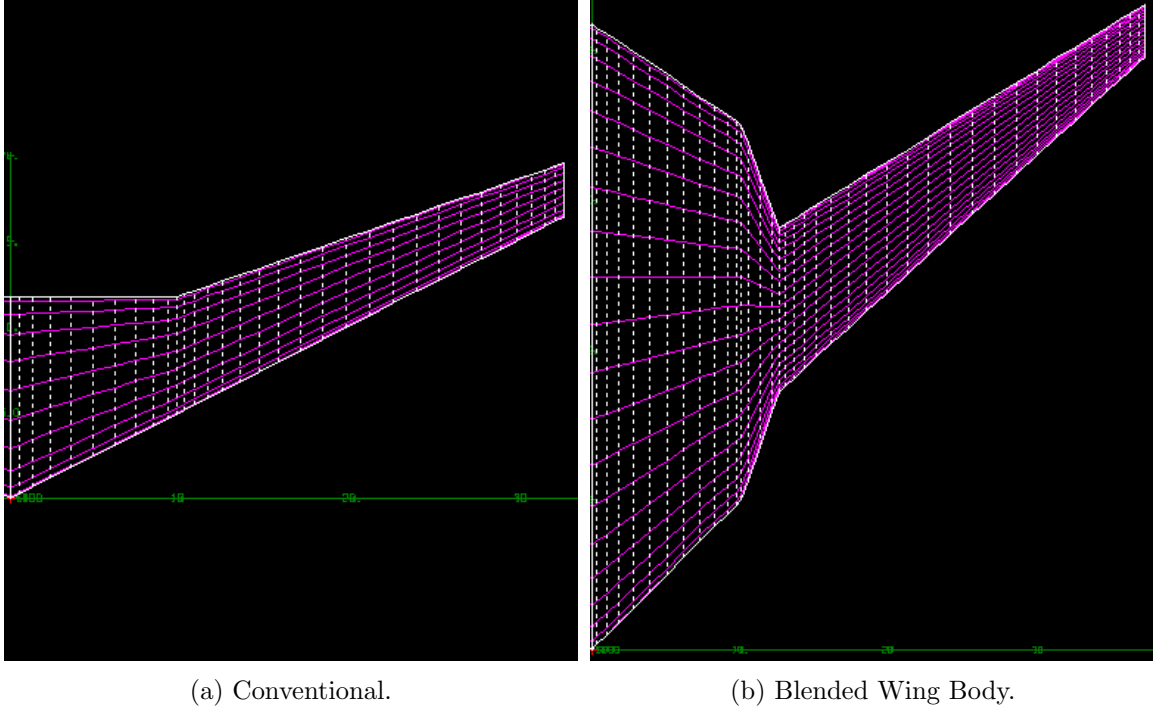


Figure 12: Panel detail.

Figure 13 shows the onset of stall for both wings.

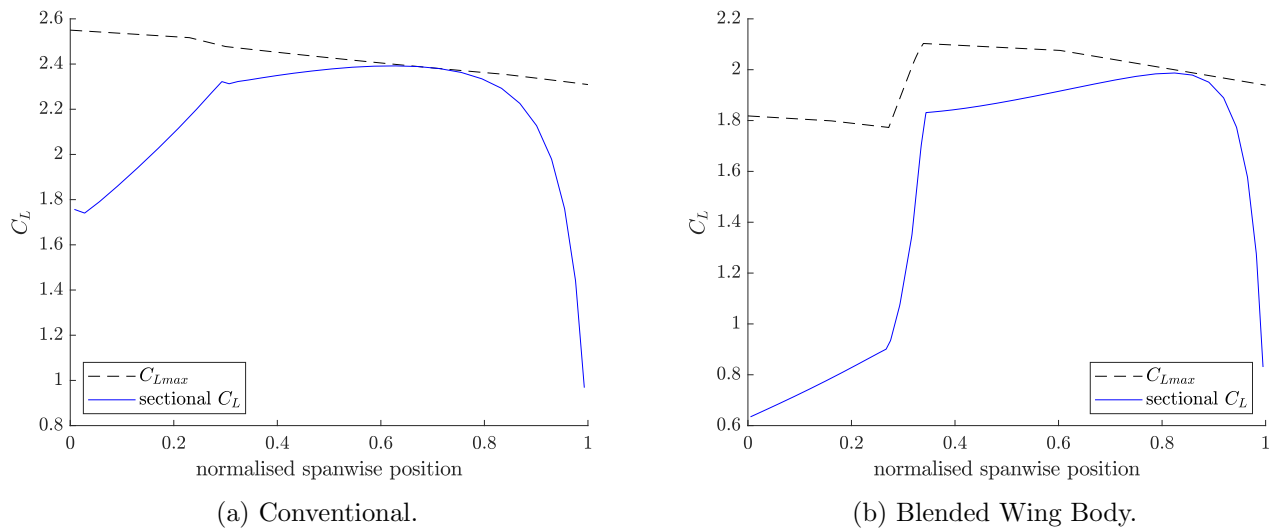
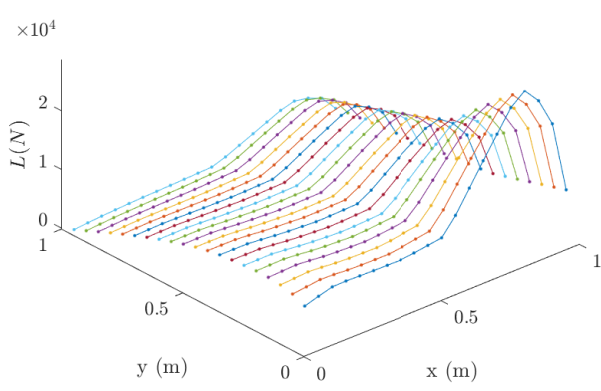
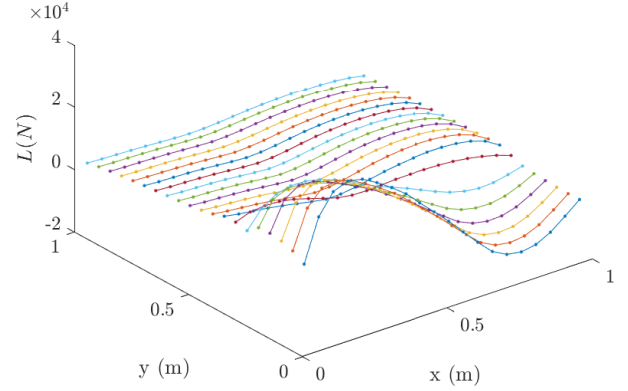


Figure 13: Stall plots.

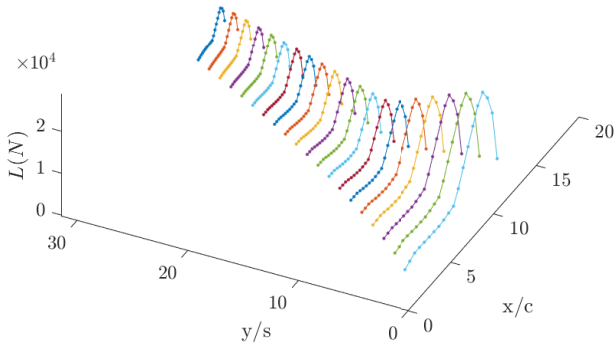
Figure 14 shows the distribution of lift along both the span and chord, using both absolute and normalised x and y coordinates. The rear loading typical of supercritical aerofoil can be clearly seen along the full span of the conventional design and in the BWB outer section. For comparison, the inner section of the BWB shows high lift at low x/c and negative lift at high x/c typical of reflex aerofoils.



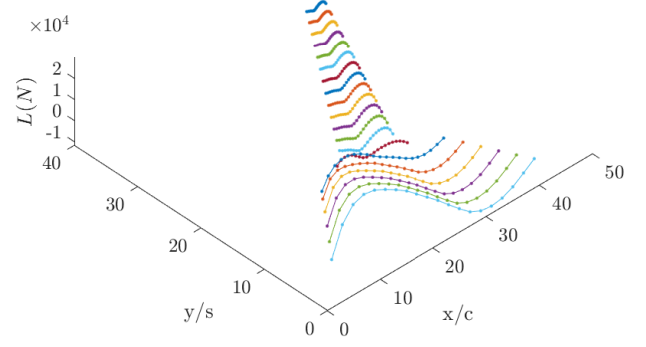
(a) Conventional, normalised.



(b) BWB, normalised.



(c) Conventional, absolute.



(d) BWB, absolute.

Figure 14: Spanwise and chordwise lift distribution.

B Code - Conventional Design

For simplicity, two sets of code were written: one for the conventional design and one for the blended wing body. In this section the conventional design code will be presented and the rough purpose of the various scripts explained. This explanation will not be done for the BWB code as it is in essence a slightly different, simpler version of the conventional design code that functions the exact same way.

Function	Description
<code>conv_main</code>	Main definition script. Defines all inputs and runs the below script.
<code>conv_multipleAnalyses</code>	Runs the below script for different types of analyses.
<code>conv_buildup</code>	Main analysis script. Runs the below scripts sequentially.
<code>conv_planformcalcs</code>	Calculates relevant planform parameters using ESDU equations.
<code>conv_planformplot</code>	Plots the current planform with the spanwise VLM strips.
<code>conv_avlin</code>	Generates input .avl file with geometry.
<code>conv_avlrun</code>	Generates input .run file with simulation parameters.
<code>conv_avlout</code>	Acquires AVL output results from respective files.
<code>conv_outproc</code>	Processes AVL output and estimates viscous drag.
<code>conv_chordwisedistrib</code>	Processes force distribution along both span and chord.
<code>conv_controlSurfaceIn</code>	Unused script. Adds sections for control surfaces to .avl input file.

B.1 conv_main

```
% Housekeeping
clear;
clc;
close all;

% Path for cmd: (ignore this, this is for manually running avl)
% D:
% cd Documents/Imperial/Y3/Subjects/"96005-GDP"/avl.analysis/avl_tree
% .\avl.exe
% load .\avl.in\conv.avl

%% Booleans
bool.plt=1; % plotting. 1 shows plots, 0 plots nothing.
bool.wingonly=1; % 1 to model only wing, 0 to model whole aircraft
bool.dhdrifudge=1; % due to dihedral, the lift and drag at the root spike up.
% if this is 1, rootmost point is neglected when
% interpolating.
bool.chordwisedistrib=0; % 1 to also find the lift distributions chordwise
bool.avldebug=1; % whether or not to show text output from avl
bool.fullanalysis=0; % if 0, only performs analysis to give desired cl. if 1, finds everything
% including lift-curve slopes, etc. and stall AoA
bool.printoutinfo=1; % 1 to print out cd, oswals, etc. (useful info for wing design)

%% File path
in.fp='D:\Documents\Imperial\Y3\Subjects\96005-GDP\avl.analysis\'; % Absolute file path to the
% folder where all these .m files are. Needed for fuselage.

%% Inputs - parametrisation
% Conventional wing divided into 3 spanwise positions. These correspond to
% root (0), end of yehudi (1) and tip (2).

%%%%%%%%% Wing %%%%%%%%%%
w.pos=[0,0,0]; % position of root LE in x,y,z space (relative to global origin)
w.S=404; % wing area
w.AR=10.3; % aspect ratio
```

```

w.ssl=0.3; % yehudi end, as %semispan
w.sweepLE=27; % leading edge sweep, deg
w.tpr=0.266; % taper ratio c2/c0
w.dhtrl=[7,5]; % dihedral for each section. Note that the span calculated is actually bigger
    than the effective span, due to this dihedral

% Angles
w.i0=0; % wing incidence, deg
w.twist=-[0,0,4]; % wing twist for positions 0,1,2. Note twist at position 1 should always be
    zero.

% Airfoils
w.afname{1}='SC(2)-0612.dat';
w.afname{2}='SC(2)-0612.dat';
w.afname{3}='SC(2)-0610.dat';

% Flight conditions
fc.M=0.8; % mach number
fc.alt=35000/3.281; % in metres
fc.iH=0; % tail incidence.

% CG position
wb.xcg=0.1; % wrt global CSYS
wb.zcg=0;

%%%%%%%%%% Hstab %%%%%%%%%%%
h.pos=[30,0,1]; %x,y,z relative to global origin. This is for the ROOT LEADING EDGE POSITION
h.S=62.1;
h.AR=10.45792;
h.tpr=0.6;
h.sweep14=30;
h.dhtrl=0;
h.afname='N0009.dat';

%%%%%%%%%% Vstab %%%%%%%%%%%
v.pos=[32,0,1.5];
v.S=48.5;
v.AR=1.5;
v.tpr=0.45;
v.sweep14=40;
v.afname='N0012.dat';

% Runtime parameters
rn.Cl=173000*9.81/(0.5*0.379597*(0.8*sqrt(1.4*287*218.808))^2*w.S); % Desired Cl

% Fuselage
fus.pos=[-10,0,0];
fus.L=50;
fus.afname='fuselage-created.dat';

%% AVL inputs
% Wing
in.Nchord=10; % number of chordwise panels
in.Cspace=1; % chordwise spacing parameter
in.Nspan=[10,22]; % number of spanwise panels - inner and outer
in.Sspace=0.6*[1,1]; % spanwise spacing parameter - inner and outer
% 0.6*[1,1] works.

% Fuselage
in.Nbody=20;
in.Bspace=1;

```

```

% Stabilisers
in.NchordH=8; % number of chordwise panels
in.CspaceH=1.0; % chordwise spacing parameter
in.NspanH=8; % number of spanwise panels - inner
in.SspaceH=1.0; % spanwise spacing parameter - inner

in.NchordV=8;
in.CspaceV=1.0;
in.NspanV=8;
in.SspaceV=1.0;

% Cheat sheet for Cspace and Sspace
%   parameter                                spacing
%   -----                                -
%
%   3.0      equal      | | | | | | | | | |
%
%   2.0      sine       || | | | | | | | |
%
%   1.0      cosine     || | | | | | | | |
%
%   0.0      equal      | | | | | | | | | |
%
%   -1.0     cosine     || | | | | | | | |
%
%   -2.0     -sine      | | | | | | | | | |
%
%   -3.0     equal      | | | | | | | | | |
%
% An intermediate parameter value will result in a blended distribution.

%% Running
[w,h,v,ot,os,oc]=conv_multipleAnalyses(bool,w,fc,wb,h,v,rn,fus,in);

```

B.2 conv_multipleAnalyses

```

function[w,h,v,ot,os,oc]=conv_multipleAnalyses(bool,w,fc,wb,h,v,rn,fus,in)
% CONV_MULTIPLEANALYSES - sets up and runs conv_buildup for the analyses
% requested.
% INPUTS: every input struct.
%   bool - booleans
%   w - wing
%   fc - flight conditions
%   wb - weight & balance
%   h - horizontal stabiliser
%   v - vertical stabiliser
%   rn - misc. avl run parameters
%   fus - fuselage
%   in - misc. avl input parameters
%
% OUTPUTS:
%   w - wing (updated)
%   h - horizontal stabiliser (updated)
%   v - vertical stabiliser (updated)
%   ot - outputs for the whole aircraft
%   os - spanwise distributions
%   oc - chordwise distributions

%% Saving backup variables
if bool.fullanalysis==1 % Saving backups if full analysis is requested (some variables change

```

```

and add/remove fields when analysis is run)
back.w=w;
back.h=h;
back.v=v;
back.rn=rn;

animateBisetion=0; % feel free to change this. 1 for animating the bisection method, 0 for
    not.
end

%% Running the main analysis
[w,h,v,ot,os,oc]=conv_buildup(bool,w,fc,wb,h,v,rn,fus,in); % runs the main analysis

%% Running full analyses
if bool.fullanalysis==1 % If full analysis is requested
    %% Setup
    % Changes bools to avoid unnecessary plotting / printing of values
    bool.chordwisedistrib=0;
    bool.plt=0;
    bool.wingonly=1;
    bool.avldebug=0;
    bool.printoutinfo=0;

    % AoAs
    aoas=[1,3]; % 2 aoas for calculating lift curve slopes

    %% Analysis case 1 - first AoA
    clear('rn');
    rn.aoa=aoas(1);
    [~,~,~,ot1,os1,~]=conv_buildup(bool,back.w,fc,wb,back.h,back.v,rn,fus,in); % runs the main
        analysis

    %% Analysis case 2 - second AoA
    rn.aoa=aoas(2);
    [~,~,~,ot2,os2,~]=conv_buildup(bool,back.w,fc,wb,back.h,back.v,rn,fus,in); % runs the main
        analysis

    %% Calculation of wing parameters
    ot.aw=(ot2.Cl-ot1.Cl)/((ot2.alpha-ot1.alpha)*pi/180); % overall wing lift curve slope in
        rad^-1
    c=ot1.Cl-ot.aw*(-ot1.Cl)*pi/180;
    ot.alpha0w=-c/ot.aw; % overall wing zero-lift aoa, in rad
    os.alocal=(os2.Cl-c-os1.Cl)/((ot2.alpha-ot1.alpha)*pi/180); % local lift curve slope

    %% STALL - spanwise Clmax distribution
    % We use xfoil data and a linear blend to find the spanwise Clmax
    % distribution.

    sss=0:0.001:1; % finer spanwise station array
    [~,il]=find(sss>=w.ssl,1); % index for the "mid" section of the planform, a.k.a. when
        yehudi ends
    cs=interp1(w.ss,w.c,sss,'linear'); % chords
    [T,~,~,rho]=atmosisa(fc.alt);
    mu=1.716e-5*(T/273)^1.5*(273+111)/(T+111); % viscosity calculation
    Res=rho*fc.M*sqrt(1.4*287*T)*cs/mu; % reynolds along span

    cd aerofoil_analysis_xfoil
    load(w.afname{1}(1:end-4)); % Grabs aero data for root airfoil
    xfoil0=thisfoil;
    load(w.afname{2}(1:end-4)); % Grabs aero data for mid airfoil
    xfoil1=thisfoil;
    load(w.afname{3}(1:end-4)); % Grabs aero data for tip airfoil

```



```

        break; % if first AoA doesn't stall, and second AoA does stall, we can continue with the
            bisection method
end

diffs=[diff1;diff2]; % cl - clmax along span for the 2 limit AoAs
precision=0.0001; % precision in the stall AoA, in degrees
difference=2*precision; % initialises difference (error)

if animateBisetion==1 % if animating
    figure;
end

while difference>precision % bisection method

    if animateBisetion==1 % if animating
        hold off;
        plot(sss,clmax,'-k')
        hold on;
        plot(sss,diffs(1,:)+clmax,'b') % plots spanwise cl variation for lower and upper AoA
        plot(sss,diffs(2,:)+clmax,'b')
    end

    aoa3=(aoas(1)+aoas(2))/2; % middle value of aoa
    rn.aoa=aoa3;
    [~,~,~,ot3,os3,~]=conv_buildup(bool,back.w,fc,wb,back.h,back.v,rn,fus,in); % runs the main
        analysis

    diff3=interp1(os.ss,os3.cl_c,sss)-clmax; % cl - clmax for the middle aoa value

    if max(diff3)>0 % if the wing has stalled somewhere along the span
        aoas(2)=aoa3; % sets middle aoa as corresponding upper/lower aoa limit
        diffs(2,:)=diff3; % assigns the difference to the upper/lower limit difference
        difference=abs(max(diffs(1,:))); % calculates how far off from convergence we are.
    else % if the wing hasn't stalled yet
        aoas(1)=aoa3;
        diffs(1,:)=diff3;
        difference=abs(max(diffs(1,:)));
    end

    if animateBisetion==1 % if animating
        plot(sss,diff3+clmax,'r') % plots for current AoA (middle of aoa bounds)
        pause(0.3); % waits to ensure user can read the plot and process it in their pea-sized
            brains
    end

end

end

ot.alphaStall=aoa3; % wing stall aoa
ot.Clmax=ot3.Cl;
 [~,ist]=max(diff3);
ot.ssStall=sss(ist); % position of first stall as percentage of semispan
end

```

B.3 conv_buildup

```

function[w,h,v,ot,os,oc]=conv_buildup(bool,w,fc,wb,h,v,rn,fus,in)
% CONV_BUILDUP - main assembly script that runs functions sequentially.
% INPUTS: every input struct.
%     bool - booleans
%     w - wing

```

```

%      fc - flight conditions
%      wb - weight & balance
%      h - horizontal stabiliser
%      v - vertical stabiliser
%      rn - misc. avl run parameters
%      fus - fuselage
%      in - misc. avl input parameters
%
% OUTPUTS:
%      w - wing (updated)
%      h - horizontal stabiliser (updated)
%      v - vertical stabiliser (updated)
%      ot - outputs for the whole aircraft
%      os - spanwise distributions
%      oc - chordwise distributions

% %% Saving variable backups
% if bool.fullanalysis==1
%     backup.w=w;
% end

%% Deletion of files
[~,~]=dos('del .\avltree\avl.in\conv.avl'); % input files
[~,~]=dos('del .\avltree\avl.in\conv.run');
[~,~]=dos('del .\avltree\avl.out\conv.fs'); % output files
[~,~]=dos('del .\avltree\avl.out\conv.ft');
[~,~]=dos('del .\avltree\avl.out\conv.fe');

% Generating the AVL files
[w,h,v]=conv_planformcalcs(w,h,v); % Calculates parameters of interest like MAC and leading
    edge positions

if bool.plt==1
    conv_planformplot(w,h,v,in,bool); % plots the planform
end

conv_avlin(w,fc,wb,in,h,v,fus,bool); % Generates .avl input file
conv_avlrun(rn,fc,bool); % Generates .run analysis file

%% Running AVL
cd avltree
if bool.avldebug==1
    dos('.\avl.exe < .\avl.in\conv.run'); % runs avl, outputting text to command window
else
    [~,~]=dos('.\avl.exe < .\avl.in\conv.run'); % runs avl without any text output
end
cd ..

%% Output Processing
[ot,os]=conv_avlout(rn,w); % Grabs the output data from text files
[ot,os]=conv_outproc(ot,os,bool,w,fc); % Processes the output data

if bool.chordwisedistrib==1 % chordwise distributions of lift
    oc=conv_chordwisedistrib(in,w,os,ot);
else
    oc=NaN;
end

end

```

B.4 conv_planformcalcs

```

function[w,h,v]=conv_planformcalcs(w,h,v)
% CONV.PLANFORMCALCS - performs some computations for planform (chords,
% MAC, etc.)
%
% INPUTS:
%   w - wing struct
%   h - horiz stabiliser struct
%   v - vert stabiliser struct
%
% OUTPUTS:
%   w - wing struct
%   h - horiz stabiliser struct
%   v - vert stabiliser struct

%% Wing
% Planform
w.b=sqrt(w.AR*w.S); % span
w.s=w.b/2; % semispan
w.ss=[0,w.ss1,1]; % spanwise stations for sections
w.y=w.ss*w.s; % y-position of each section
w.xLE=w.y*tand(w.sweepLE); % x-pos of leading edge of each section
w.c(1)=(w.S+w.y(3)*w.y(2)*tand(w.sweepLE))/(w.y(2)*(1-w.tpr)+w.y(3)*(1+w.tpr)); % root chord
    calculation, based on expressing wing area as function of root chord
w.c=[w.c(1),w.c(1)-w.xLE(2),w.c(1)*w.tpr]; % chord of each section
w.xTE=w.xLE+w.c; % TE positions
w.z=[0,w.y(2)*sind(w.dhtrl(1)),w.y(2)*sind(w.dhtrl(1))+(w.y(3)-w.y(2))*sind(w.dhtrl(2))]; %
    vertical positions (model dihedral)

% MAC
a=(w.c(2:end)-w.c(1:end-1))./(w.y(2:end)-w.y(1:end-1)); % for each section, c(y) = ay + b
b=w.c(1:end-1)-a.*w.y(1:end-1);
w.cbar=(2/w.S)*sum(a.^2.*(w.y(2:end).^3-w.y(1:end-1).^3)/3+a.*b.*(w.y(2:end).^2-w.y(1:end-1).^2)+b.^2.*(w.y(2:end)-w.y(1:end-1))); % MAC

if w.c(1)==w.c(2) && w.c(2)==w.c(3) % for validation purposes
    w.ybar=0;
    w.xLEbar=0;
else % under normal scenarios
    w.ybar=interp1(w.c,w.y,w.cbar,'linear');
    w.xLEbar=interp1(w.c,w.xLE,w.cbar,'linear');
end
w.i=w.i0+w.twist; % incidence of each section.

%% Hstab
h.ss=[0,1];
h.twist=[0,0];
h.b=sqrt(h.AR*h.S); % span
h.s=h.b/2; % semispn
h.c0=h.S/h.s/(1+h.tpr); % root chord
h.c=h.c0*[1,h.tpr]; % chord array
h.y=[0,h.s]; % y-position array
h.z=[0,h.s*sind(h.dhtrl)]; % z-position array (dihedral)
h.sweepLE=atan((tand(h.sweep14)*h.s+(h.c(1)-h.c(2))/4)/h.s)*180/pi; % leading edge sweep
h.xLE=[0,h.s*tand(h.sweepLE)]; % LE x-position array
h.xTE=h.xLE+h.c; % TE

h.cbar=2*h.c0/3*(1+h.tpr+h.tpr^2)/(1+h.tpr); % MAC
h.ybar=interp1(h.c,h.y,h.cbar); % y location of MAC
h.xLEbar=h.ybar*tand(h.sweepLE); % x location of MAC LE

%% Vstab (note this then gets rotated, so the {y,z} coordinates here transform to {z,y}
    coordinates
v.ss=[0,1];

```

```

v.twist=[0,0];
v.b=sqrt(v.AR*v.S); % span
v.s=v.b;
v.c0=2*v.S/v.b/(1+v.tpr); % root chord
v.c=v.c0*[1,v.tpr]; % chord array
v.y=[0,v.b]; % y-pos array
v.z=[0,0]; % z-pos array
v.sweepLE=atan((tand(v.sweep14)*v.b+(v.c(1)-v.c(2))/4)/v.b)*180/pi; % LE sweep
v.xLE=[0,v.b*tand(v.sweepLE)]; % LE position
v.xTE=v.xLE+v.c; % TE position

v.cbar=2*v.c0/3*(1+v.tpr+v.tpr^2)/(1+v.tpr); % MAC
v.ybar=interp1(v.c,v.y,v.cbar); % y location of MAC
v.xLEbar=v.ybar*tand(v.sweepLE); % x location of MAC

%% Misc
x14=w.xLE+w.c/4;
w.sweep14=atan((x14(2:end)-x14(1:end-1))./(w.y(2:end)-w.y(1:end-1)))*180/pi;
w.xlffoffset=w.xLE+w.c/4-w.c(1)/4;
end

```

B.5 conv_planformplot

```

function[]=conv_planformplot(w,h,v,in,bool)
% CONV_PLANFORMPLOT - Plots the planform on the left (with panels), and
% zooms in on the kink on the right (to check the panel spacing at the kink)
%
% INPUTS (structs):
%   w - wing
%   h - h stab
%   v - v stab
%   in - avl input file
%   bool - boolean

figure('Position',[ (1920-1200)/2, (1080-600)/2, 1200, 600]); % creates figure

for ii=1:2 % plots the full planform on both sides of the figure
    subplot(1,2,ii);
    hold on;

    % Wing
    plot(w.pos(1)+w.xLE,w.y,'-k'); % LE
    plot(w.pos(1)+w.xLE+w.c/4,w.y,':k'); % 1/4 chord line
    plot(w.pos(1)+w.xTE,w.y,'-k'); % TE
    plot(w.pos(1)+[w.xLE(1),w.xTE(1)],w.y(1)*[1,1],'-k'); % Root chord
    plot(w.pos(1)+[w.xLE(2),w.xTE(2)],w.y(2)*[1,1],'-k'); % Mid chord
    plot(w.pos(1)+[w.xLE(3),w.xTE(3)],w.y(3)*[1,1],'-k'); % Tip chord

    plot(w.pos(1)+w.xLEbar+[0,w.cbar],w.ybar*[1,1], '--k'); % MAC

    % Panels
    ssp=[spacing(in.Nspan(1),in.Sspace(1))*w.ss(2),w.ss(2)+spacing(in.Nspan(2),in.Sspace(2))*(
        w.ss(3)-w.ss(2))]; % spanwise spacing of panels
    xlep=interp1(w.ss,w.xLE,ssp,'linear'); % interpolating leading edge position
    cp=interp1(w.ss,w.c,ssp,'linear'); % interpolating chords

    for ii=1:length(cp) % for each panel edge
        plot(w.pos(1)+xlep(ii)+[0,cp(ii)],ssp(ii)*w.s*[1,1],':b'); % plots chord for panel edge
    end
end

```

```

if bool.wingonly==0 % If running also the hstab and vstab
    % Hstab
    plot(h.pos(1)+h.xLE,h.y, '-k'); % LE
    plot(h.pos(1)+h.xLE+h.c/4,h.y, ':k'); % 1/4 chord line
    plot(h.pos(1)+h.xTE,h.y, '-k'); % TE
    plot(h.pos(1)+[h.xLE(1),h.xTE(1)],h.y(1)*[1,1], '-k'); % Root chord
    plot(h.pos(1)+[h.xLE(2),h.xTE(2)],h.y(2)*[1,1], '-k'); % Mid chord
    plot(h.pos(1)+h.xLEbar+[0,h.cbar],h.ybar*[1,1], '--k'); % Tip chord

    % Vstab
    plot(v.pos(1)+v.xLE,-v.y, '-b'); % LE
    plot(v.pos(1)+v.xLE+v.c/4,-v.y, ':b'); % 1/4 chord line
    plot(v.pos(1)+v.xTE,-v.y, '-b'); % TE
    plot(v.pos(1)+[v.xLE(1),v.xTE(1)],-v.y(1)*[1,1], '-b'); % Root chord
    plot(v.pos(1)+[v.xLE(2),v.xTE(2)],-v.y(2)*[1,1], '-b'); % Mid chord
    plot(v.pos(1)+v.xLEbar+[0,v.cbar],-v.ybar*[1,1], '--b'); % Tip chord
end

xlabel('x (m)');
ylabel('y (m)');
axis equal;
title('Planform'); % title for left plot
end

title('Panel detail at kink'); % title for right plot

% Changing limits on the plot
xlim(w.xLE(2)+[0,w.c(2)]+0.1*w.c(2)*[-1,1]);
ylim(w.y(2)+[-1,1]*w.s*0.1);
end

```

B.6 conv_avlin

```

function[]=conv_avlin(w,fc,wb,in,h,v,fus,bool)
% CONVAVLIN - generates AVL input file for conventional aircraft
%
% INPUTS: a series of structs
%     w - wing
%     fc - flight conditions
%     wb - weight and balance
%     in - miscellaneous avl inputs for the .avl file
%     h - horizontal stabiliser
%     v - vertical stabiliser
%     bool - booleans (whether or not to plot, whether or not to include
%     tail in analysis, etc)
%
% OUTPUTS: none

%% Grabbing airfoil coordinates from files
cd aerofoil_analysis_xfoil % changes to airfoil directory

% Wing, root
fid=fopen(w.afname{1}, 'r+');
w.afCoords0=fscanf(fid, '%f', [2,inf]); % takes coordinates from file and stores in an array
fclose(fid);

% Wing, mid
fid=fopen(w.afname{2}, 'r+');
w.afCoords1=fscanf(fid, '%f', [2,inf]);
fclose(fid);

```

```

% Wing, tip
fid=fopen(w.afname{3}, 'r+');
w.afCoords2=fscanf(fid, '%f', [2,inf])';
fclose(fid);

% Hstab
fid=fopen(h.afname, 'r+');
h.afCoords=fscanf(fid, '%f', [2,inf])';
fclose(fid);

% Vstab
fid=fopen(v.afname, 'r+');
v.afCoords=fscanf(fid, '%f', [2,inf])';
fclose(fid);

cd ..

%% Setup
fid=fopen('conv.avl', 'w+'); % creates the input file

fprintf(fid, 'GDP_Conventional\n');
fprintf(fid, '%.4f                                ! Mach\n', fc.M); % mach #
fprintf(fid, '0      0      0.0                    ! iYsym iZsym Zsym\n'); % defining planes of
    symmetry
fprintf(fid, '%.4f    %.4f    %.4f                ! Sref    Cref    Bref\n', w.S, w.cbar, w.b); %
    reference values
fprintf(fid, '%.4f    %.4f    %.4f                ! Xref    Yref    Zref\n', wb.xcg, 0, wb.zcg); % cg
    position
fprintf(fid, '0.020                                ! CDp    (optional)\n'); % default profile drag
    coefficient added to geometry

fprintf(fid, '#\n');
fprintf(fid, '#-----\n');
fprintf(fid, '#\n');

%% Wing
fprintf(fid, 'SURFACE                                ! (keyword)\n');
fprintf(fid, '    Wing                                ! surface name string\n');
fprintf(fid, '    %d    %.4f                                ! Nchord  Cspace\n', in.Nchord, in.Cspace); %
    chordwise panel definition
fprintf(fid, '#\n');
fprintf(fid, '    SCALE                                ! (keyword)\n'); % sets scale to 1
fprintf(fid, '    1.0    1.0    1.0                            ! Xscale  Yscale  Zscale\n');
fprintf(fid, '#\n');
fprintf(fid, '    TRANSLATE                                ! (keyword)\n'); % moves wing to respective
    position
fprintf(fid, '    %.4f    %.4f    %.4f                            ! dX   dY   dZ\n', w.pos(1), w.pos(2), w.pos(3));
fprintf(fid, '#\n');
fprintf(fid, '    ANGLE                                ! (keyword)\n'); % sets wing incidence
fprintf(fid, '    %.4f                                ! dAinc\n', w.i0);
fprintf(fid, '#\n');
fprintf(fid, '    YDUPLICATE                                ! (keyword)\n'); % generates the other side of
    the wing
fprintf(fid, '    0.0                                ! Ydupl\n');
fprintf(fid, '#\n');
fprintf(fid, '#\n');

% Root section
fprintf(fid, '#    (root section)\n');
fprintf(fid, '    SECTION                                ! (keyword)\n');
fprintf(fid, '    %.4f    %.4f    %.4f    %.4f    %.4f    %.4f    %.4f    ! Xle Yle Zle  Chord
    Ainc    [ Nspan Sspace ]\n', w.xLE(1), w.y(1), w.z(1), w.c(1), w.twist(1), in.Nspan(1), in.Sspace

```

```

(1)); % geometry
fprintf(fid, '      AIRFOIL      0.0    1.0      ! (keyword)    [ optional x/c range ]\n'); %
    airfoil
fprintf(fid, '      %s                      ! filename string\n', ['D:\Documents\Imperial\
    Y3\Subjects\96005-GDP\avl_analysis\avl_tree\' ,w.afname0]); % must be absolute file path.
    Must change this.
fprintf(fid, '      %f      %f\n', w.afCoords0');
fprintf(fid, '#\n');

% Mid section
fprintf(fid, '#      (mid section)\n');
fprintf(fid, '      SECTION                      ! (keyword)\n');
fprintf(fid, '      %.4f    %.4f    %.4f    %.4f    %.4f    %.4f    %.4f    ! Xle Yle Zle    Chord
    Ainc    [ Nspan Sspace ]\n', w.xLE(2), w.y(2), w.z(2), w.c(2), w.twist(2), in.Nspan(2), in.Sspace
    (2));
fprintf(fid, '      AIRFOIL      0.0    1.0      ! (keyword)    [ optional x/c range ]\n');
fprintf(fid, '      %s                      ! filename string\n', ['D:\Documents\Imperial\
    Y3\Subjects\96005-GDP\avl_analysis\avl_tree\' ,w.afname1]);
fprintf(fid, '      %f      %f\n', w.afCoords1');
fprintf(fid, '#\n');

% Tip section
fprintf(fid, '#      (tip section)\n');
fprintf(fid, '      SECTION                      ! (keyword)\n');
fprintf(fid, '      %.4f    %.4f    %.4f    %.4f    %.4f    ! Xle Yle Zle    Chord Ainc    [ Nspan
    Sspace ]\n', w.xLE(3), w.y(3), w.z(3), w.c(3), w.twist(3));
fprintf(fid, '      AIRFOIL      0.0    1.0      ! (keyword)    [ optional x/c range ]\n');
fprintf(fid, '      %s                      ! filename string\n', ['D:\Documents\Imperial\
    Y3\Subjects\96005-GDP\avl_analysis\avl_tree\' ,w.afname2]);
fprintf(fid, '      %f      %f\n', w.afCoords2');

% Can add sections for flaps, ailerons, etc.

fprintf(fid, '#\n');
fprintf(fid, '#-----\n');
fprintf(fid, '#\n');

if bool.wingonly==0 % only add other surfaces if needed
    %% Hstab
    fprintf(fid, 'SURFACE                      ! (keyword)\n');
    fprintf(fid, '      Hstab                      ! surface name string\n');
    fprintf(fid, '      %d    %.4f    %d    %.4f          ! Nchord Cspace Nspan Sspace\n',
        in.NchordH, in.CspaceH, in.NspanH, in.SspaceH); % panel definition
    fprintf(fid, '#\n');
    fprintf(fid, '      SCALE                      ! (keyword)\n'); % set scale to 1
    fprintf(fid, '      1.0    1.0    1.0          ! Xscale Yscale Zscale\n');
    fprintf(fid, '#\n');
    fprintf(fid, '      TRANSLATE                  ! (keyword)\n'); % moving to correct
        position
    fprintf(fid, '      %.4f    %.4f    %.4f          ! dX dY dZ\n', h.pos(1), h.pos(2), h.pos(3))
        ;
    fprintf(fid, '#\n');
    fprintf(fid, '      ANGLE                      ! (keyword)\n'); % sets incidence
    fprintf(fid, '      %.4f                      ! dAinc\n', fc.iH);
    fprintf(fid, '#\n');
    fprintf(fid, '      YDUPLICATE                  ! (keyword)\n'); % generates the other
        side
    fprintf(fid, '      0.0                      ! Ydupl\n');
    fprintf(fid, '#\n');

% Root
fprintf(fid, '#      (root section)\n');

```



```

fprintf(fid, '      SECTION                                ! (keyword)\n');
fprintf(fid, '      %.4f  %.4f  %.4f  %.4f  %.4f  ! Xle Yle Zle  Chord Ainc\n',
      h.xLE(1),h.y(1),h.z(1),h.c(1),0); % geometry
fprintf(fid, '      AIRFOIL      0.0  1.0      ! (keyword)  [ optional x/c range ]\n');
      % airfoil
% fprintf(fid, '      %s                                ! filename string\n', ['D:\Documents\
Imperial\Y3\Subjects\96005_GDP\avl_analysis\avl_tree\',h.afname]); % abs file path
fprintf(fid, '      %f      %f\n',h.afCoords');
fprintf(fid, '#\n');

% Tip
fprintf(fid, '#      (tip section)\n');
fprintf(fid, '      SECTION                                ! (keyword)\n');
fprintf(fid, '      %.4f  %.4f  %.4f  %.4f  %.4f  ! Xle Yle Zle  Chord Ainc\n',
      h.xLE(2),h.y(2),h.z(2),h.c(2),0);
fprintf(fid, '      AIRFOIL      0.0  1.0      ! (keyword)  [ optional x/c range ]\n');
% fprintf(fid, '      %s                                ! filename string\n', ['D:\Documents\
Imperial\Y3\Subjects\96005_GDP\avl_analysis\avl_tree\',h.afname]);
fprintf(fid, '      %f      %f\n',h.afCoords');
fprintf(fid, '#\n');

fprintf(fid, '#\n');
fprintf(fid, '#-----\n');
fprintf(fid, '#\n');

%% Vstab
fprintf(fid, 'SURFACE                                ! (keyword)\n');
fprintf(fid, '      Vstab                                ! surface name string\n');
fprintf(fid, '      %d  %.4f  %d  %.4f                                ! Nchord Cspace Nspan Sspace\n',
      in.NchordV,in.CspaceV,in.NspanV,in.SspaceV); % panel definition
fprintf(fid, '#\n');
fprintf(fid, '      SCALE                                ! (keyword)\n'); % sets scale to 1
fprintf(fid, '      1.0  1.0  1.0                                ! Xscale Yscale Zscale\n');
fprintf(fid, '#\n');
fprintf(fid, '      TRANSLATE                                ! (keyword)\n'); % moves to position
fprintf(fid, '      %.4f  %.4f  %.4f                                ! dX dY dZ\n',v.pos(1),v.pos(2),v.pos(3))
;
fprintf(fid, '#\n');
fprintf(fid, '      ANGLE                                ! (keyword)\n'); % zero incidence
fprintf(fid, '      0.0                                ! dAinc\n');
fprintf(fid, '#\n');

% Root
fprintf(fid, '#      (root section)\n');
fprintf(fid, '      SECTION                                ! (keyword)\n');
fprintf(fid, '      %.4f  %.4f  %.4f  %.4f  %.4f  ! Xle Yle Zle  Chord Ainc\n',
      v.xLE(1),v.z(1),v.y(1),v.c(1),0); % geometry
fprintf(fid, '      AIRFOIL      0.0  1.0      ! (keyword)  [ optional x/c range ]\n');
      % airfoil
% fprintf(fid, '      %s                                ! filename string\n', ['D:\Documents\
Imperial\Y3\Subjects\96005_GDP\avl_analysis\avl_tree\',v.afname]); % absolute path
fprintf(fid, '      %f      %f\n',v.afCoords');
fprintf(fid, '#\n');

% Tip
fprintf(fid, '#      (tip section)\n');
fprintf(fid, '      SECTION                                ! (keyword)\n');
fprintf(fid, '      %.4f  %.4f  %.4f  %.4f  %.4f  ! Xle Yle Zle  Chord Ainc\n',
      v.xLE(2),v.z(2),v.y(2),v.c(2),0);
fprintf(fid, '      AIRFOIL      0.0  1.0      ! (keyword)  [ optional x/c range ]\n');
% fprintf(fid, '      %s                                ! filename string\n', ['D:\Documents\
Imperial\Y3\Subjects\96005_GDP\avl_analysis\avl_tree\',v.afname]);
fprintf(fid, '      %f      %f\n',v.afCoords');

```

```

fprintf(fid, '#\n');
fprintf(fid, '#\n');
fprintf(fid, '#-----\n');
fprintf(fid, '#\n');

%% Fuselage
fprintf(fid, 'BODY                ! (keyword)\n');
fprintf(fid, '    Fuselage        ! surface name string\n');
fprintf(fid, '    %d    %.4f                ! Nbody Bspace\n', in.Nbody, in.Bspace); %
    panel definition
fprintf(fid, '#\n');
fprintf(fid, '    SCALE                    ! (keyword)\n');
fprintf(fid, '    %.4f    %.4f    %.4f        ! Xscale Yscale Zscale\n', fus.L, fus.L,
    fus.L); % scales to full length
fprintf(fid, '#\n');
fprintf(fid, '    TRANSLATE                ! (keyword)\n'); % moves to position
fprintf(fid, '    %.4f    %.4f    %.4f        ! dX dY dZ\n', fus.pos(1), fus.pos(2),
    fus.pos(3));
fprintf(fid, '#\n');
fprintf(fid, '    BFILE    0.0    1.0        ! (keyword)\n'); % fuselage "airfoil" file
fprintf(fid, '    %s                ! filename string\n', [in.fp, fus.afname]); %
    abs file path
fprintf(fid, '#\n');

end

fclose(fid); % closes file

%% Moving file to appropriate folder
copyfile('conv.avl', 'avl_tree/avl_in');
delete('conv.avl');
end

```

B.7 conv_avlrun

```

function []=conv_avlrun(rn,fc,bool)
% CONV_AVLRUN - generates AVL run file for conventional aircraft
%
% INPUTS - structs:
%   rn - misc AVL inputs for the .run file
%   fc - flight Conditions
%   bool - booleans, for eg. for plotting or not

%% Setup
cd avl_tree/avl_in % moves to input folder
fid=fopen('conv.run','w+'); % creates the input file

% Air density
[~,~,~,rho]=atmosisa(fc.alt);

%% Writing
fprintf(fid, 'LOAD %s \n', '\avl_in\conv.avl'); % loads geometry from .avl file
% fprintf(fid, 'OPER \n');
% fprintf(fid, 'G \n');
% fprintf(fid, 'CA \n');
fprintf(fid, 'OPER \n');

if isfield(rn, 'Cl') % if we are running a specific cl analysis
    fprintf(fid, 'Cl \n');
    fprintf(fid, 'C %.4f \n', rn.Cl); % target CL. Will run an analysis where AoA changes
end

```

```

        until desired Cl is reached
        fprintf(fid, 'D %.4f \n', rho); % density
        fprintf(fid, '\n');
elseif isfield(rn, 'aoa') % if we are running a specific aoa analysis
    fprintf(fid, 'A A %.4f \n', rn.aoa); % target CL. Will run an analysis where AoA changes
        until desired Cl is reached
end

fprintf(fid, 'X \n'); % runs the analysis

% Save File for Total Force
fprintf(fid, 'FT\n');
fprintf(fid, '%s \n', './avl.out\conv.ft');
%fprintf(fid, 'O \n');
%fprintf(fid, '\n');

% Save File for Strip Force (spanwise lift/drag distributions)
fprintf(fid, 'FS\n');
fprintf(fid, '%s \n', './avl.out\conv.fs');
%fprintf(fid, 'O \n');
%fprintf(fid, '\n');

if bool.chordwisedistrib==1
    % Save File for Element Force (spanwise and chordwise lift distribution)
    fprintf(fid, 'FE\n');
    fprintf(fid, '%s \n', './avl.out\conv.fe');
    %fprintf(fid, 'O \n');
    %fprintf(fid, '\n');
end

fprintf(fid, '\n');
fclose(fid); % closes file
cd ..
cd ..
end

```

B.8 conv_avlout

```

function[ot,os]=conv_avlout(rn,w)
% CONVAVLOUT - gets output from avl analysis output files
%
% INPUTS (structs)
%     rn - miscellaneous avl inputs for the .run file
%     w - wing

%% Copy the files to current folder
%cd avl.tree\avl.out % moves to respective folder
copyfile('avl.tree\avl.out\conv.fs','conv.fs');
copyfile('avl.tree\avl.out\conv.ft','conv.ft');
% if bool.chordwisedistrib==1
%     copyfile('avl.tree\avl.out\conv.fe','conv.fe');
% end

%% Reading Total forces
fid=fopen('conv.ft','r+');

ot.alpha=myAvlFileRead(fid,'Alpha'); % AoA
ot.Cl =myAvlFileRead(fid,'CLtot'); % Cl

```

```

ot.Cdi =myAvlFileRead(fid,'CDff'); % lift=induced Cd
ot.e =myAvlFileRead(fid,'e'); % oswaldds

fclose(fid);

%% Reading Strip forces
fid=fopen('conv.fs','r+');

myAvlFileRead(fid,'cm_c/4');
data=fscanf(fid,'%f',[13,inf]); % scans full data table
fclose(fid);

% Processing of strip forces
os.y=data(2,:); % y-positions
os.c=data(3,:); % chords
ccl=data(5,:); % cl * section chord
os.cl_cref=ccl/w.cbar; % cl, using MAC as reference chord
os.cl_c=ccl./os.c; % cl, using section chord as reference chord
os.alphai=-data(6,:); % induced AoAs
os.cdi=data(9,:); % induced Cd
%os.cm14=data(11,:);

%% Delete the temporary files in this folder
delete('conv.fs');
delete('conv.ft');
% if bool.chordwisedistrib==1
%     delete('conv.fe');
% end
% cd ..
% cd ..
end

```

B.9 conv_outproc

```

function[ot,os]=conv_outproc(ot,os,bool,w,fc)
% CONV_OUTPROC - processes the output, estimates viscous drag, etc
%
% INPUTS - structs:
%     ot - total aero data (cl, cdi, e)
%     os - spanwise aero distributions
%     bool - boolean struct
%     w - wing
%     fc - flight conditions
%
% OUTPUTS - structs
%     ot - total aero data (cl, cdi, e)
%     os - spanwise aero distributions

%% Processing and smooth spanwise distributions
fi=bool.dhdr1fudge+1; % 1 if no fudge, 2 if fudge (skips rootmost point in distribution)

os.ss=os.y/w.s; % spanwise stations - 0 at root, 1 at tip
sss=0:0.001:1; % finer spanwise station array
cls=interp1(os.ss(fi:end),os.cl_cref(fi:end),sss,'pchip'); % interpolating cl
cdi=interp1(os.ss(fi:end),os.cdi(fi:end),sss,'pchip'); % interpolating cdi

[~,i1]=find(sss>=w.ss1,1); % index for the "mid" section of the planform, a.k.a. when yehudi
ends
is=interp1(w.ss,w.i,sss,'linear'); % incidences along span

```

```

as=is+ot.alpha+interp1(os.ss(fi:end),os.alphai(fi:end),sss,'pchip'); % "angles of attack"

%% Viscous drag estimation
cs=interp1(w.ss,w.c,sss,'linear');

[T,~,~,rho]=atmosisa(fc.alt);
mu=1.716e-5*(T/273)^1.5*(273+111)/(T+111); % viscosity calculation
Res=rho*fc.M*sqrt(1.4*287*T)*cs/mu;

cd aerofoilanalysis_xfoil
load(w.afname{1}(1:end-4)); % Grabs aero data for root airfoil
xfoil0=thisfoil;
load(w.afname{2}(1:end-4)); % Grabs aero data for mid airfoil
xfoil1=thisfoil;
load(w.afname{3}(1:end-4)); % Grabs aero data for tip airfoil
xfoil2=thisfoil;
cd ..

% Blending airfoil data
% cd0=interp1(xfoil0.a,xfoil0.cd,as,'pchip'); % visc. drag for root airfoil
% cd1=interp1(xfoil1.a,xfoil1.cd,as,'pchip'); % mid
% cd2=interp1(xfoil2.a,xfoil2.cd,as,'pchip'); % tip

cd0=interp2(xfoil0.re,xfoil0.a,xfoil0.cd,Res,as); % visc. drag for root airfoil
cd1=interp2(xfoil1.re,xfoil1.a,xfoil1.cd,Res,as); % mid
cd2=interp2(xfoil2.re,xfoil2.a,xfoil2.cd,Res,as); % tip

cdv=(cd0(1:i1).*(sss(i1)-sss(1:i1))+cd1(1:i1).*sss(1:i1))/sss(i1); % blending for inner section
cdv=[cdv,(cd1(i1+1:end).*(1-sss(i1+1:end))+cd2(i1+1:end).*(sss(i1+1:end)-sss(i1)))/(1-sss(i1))
]; % and outer section

% Definitions
labelSize=14;
legendSize=12;
%titleSize=20;
%sgtitleSize=24;
tickSize=12;

% figure; % this figure shows how the blending along the span is happening
% hold on;
% plot(sss,cd0,'-b');
% plot(sss,cd1,'-r');
% plot(sss,cd2,'-g');
% plot(sss,cdv,'-k'); %%%%%%%%%%%%%%%
% plot(w.ss1*[1,1],[5.6,6.2]*1e-3,'--k');
% legend('root','mid','tip','blended','Location','northwest','Interpreter','latex','FontSize',
%     legendSize);
% xlabel('normalised spanwise position','Interpreter','latex','FontSize',labelSize);
% ylabel('Viscous $C_{D}$','Interpreter','latex','FontSize',labelSize);
% %title('Blending of airfoil sections spanwise');
% %figure;
%
% ylim([5.6,6.2]*1e-3);
% set(gca,'TickLabelInterpreter','latex');
% ax=gca;
% ax.FontSize=tickSize;

%% Elliptical estimation - elliptical Cl that gives the same total lift
eldis=sqrt(1-sss.^2);
cls_ellip=eldis*trapz(sss,cls)/trapz(sss,eldis);
%cls_ellip=eldis*(w.S*ot.Cl/w.s/w.cbar/2)/trapz(sss,eldis);

```

```

%% Final computations
% Here, we will define Kl Kd Kdi:
% D = 0.5 rho V^2 S Cd
% Kd = S Cd = spanwise integral of c Cd

% % Ignore this block - it was used for testing/validation purposes
% Kl=trapz(sss*w.s,cls)*w.cbar % from our fine, interpolated distribution
% Kl=trapz(sss*w.s,cls_ellip)*w.cbar % from elliptical
% Kl=trapz(os.y,os.cl_c.*os.c) % from coarse, avl-outputted cl distribution (normalised with
    sectional chord)
% Kl=trapz(os.y,os.cl_cref)*w.cbar % from coarse, avl-outputted cl distribution (normalised
    with MAC)
% Kl=ot.Cl*w.S/2 % From total Cl required
%
% Kdi=trapz(sss*w.s,cdis.*cs) % from our fine, interpolated distribution
% Kdi=ot.Cdi*w.S/2 % from our coarse, avl-outputted cdi distribution
% % Note that these 2 do not match. This is because the bottom one uses total
% % Cdi from trafftz plane (which is more accurate). The top one
% % (distribution) uses Cdi directly from the VLM. So everything is fine

Kdv=trapz(sss*w.s,cdv.*cs)*2; % integrates viscous drag along the wing
ot.Cdv=Kdv/w.S; % total viscous drag

ot.Cd=ot.Cdi+ot.Cdv; % total total drag

% Recall, of course, that
% Cl_c = lift / q c
% Cl_Cref = lift / q cref
%
% L = q S Cl = q cbar int(Cl_cref)
%         = q int (Cl_c * c)

%% Printing out results
if bool.printoutinfo==1
    disp(['AoA = ',num2str(ot.alpha),' (angle of attack)']);
    %disp(['Cl = ',num2str(ot.Cl),' (lift)']);
    disp(['e = ',num2str(ot.e),' (oswalds efficiency)']);
    %disp(['Cdv = ',num2str(ot.Cdv),' (viscous drag)']);
    %disp(['Cdi = ',num2str(ot.Cdi),' (induced drag)']);
    disp(['Cd = ',num2str(ot.Cd),' (total drag)']);
    disp(['Cdv was ',num2str(ot.Cdv/ot.Cd*100),'% of Cd']);
    disp(['Cdi was ',num2str(ot.Cdi/ot.Cd*100),'% of Cd']);
    disp(['L/D = ',num2str(ot.Cl/ot.Cd),' (for wing only)']);
end

%% Plotting
if bool.plt==1
    figure;
    subplot(1,2,1); % Cl-span graph
    hold on;
    plot(os.ss,os.cl_cref,'xk','HandleVisibility','off'); % AVL data
    plot(sss,cls,'-k'); % smoothed
    plot(sss,cls_ellip,'-b'); % elliptical
    xlabel('y/s');
    ylabel('Cl');
    legend('avl','elliptical');
    %plot(os.ss,os.cl_cref(1)*sqrt(1-os.ss.^2));

    subplot(1,2,2); % Cd-span graph (not very useful or accurate)
    hold on;
    plot(sss,cdv+cdis,'-k'); % total drag
    plot(sss,cdv,'-b'); % viscous

```

```

plot(os.ss,os.cdi,'xr','HandleVisibility','off'); % lift-induced
plot(sss,cdis,'-r');
%plot(os.ss,os.cl_cref.*sind(os.alphai),'-g')
%plot(os.ss,os.cl_c.*sind(os.alphai),'-m')
xlabel('y/s');
ylabel('Cd');
legend('total','viscous','lift ind');
sgtitle('Spanwise aero');
end
end

```

B.10 conv_chordwisedistrib

```

function[oc]=conv_chordwisedistrib(in,w,os,ot)
% CONV_CHORDWISEDISTRIB - finds the chordwise (and spanwise) distribution
% of lift
%
% INPUTS (structs):
%   in - avl misc. inputs
%   w - wing
%   os and ot (not needed, but are used in some commented-out sections.
%
% OUTPUT:
%   oc - chordwise distribution struct

%% Initial thoughts
Nc=in.Nchord;
Ns=sum(in.Nspan);

% variable initialisations
oc.x=zeros(Nc,Ns);
oc.y=zeros(1,Ns);
oc.dCp=oc.x;
oc.dx=oc.x;
oc.swidth=oc.y; % strip width

%% Reading output file

copyfile('avl-tree/avl-out/conv.fe','conv.fe');
fid=fopen('conv.fe','r+');
myAvlFileRead(fid,'
*****');

for ii=1:Ns
    oc.swidth(ii)=myAvlFileRead(fid,'Width'); % strip width
    myAvlFileRead(fid,'DX');
    data=fscanf(fid,'%f',[7,inf]); % full data stable for this strip
    oc.x(:,ii)=data(:,2);
    oc.y(ii)=data(1,3);
    oc.dx(:,ii)=data(:,5);
    oc.dCp(:,ii)=data(:,7);
end

fclose(fid);
delete('conv.fe');
% cd ..
% cd ..

%% Calculations - remember spanwise goes horizontally, chordwise goes down

%oc.F-q=oc.dCp.*oc.dx.*oc.swidth; % this sums to give total Kl this is essentially force (lift)

```

```

    / dynamic head
oc.F_q_2d=oc.dCp.*oc.dx; % Basically, force distribution.

% figure; % Plots F_q_2d to show
% for ii=1:Ns
%     plot3(oc.x(:,ii),oc.y(ii)*ones(Nc,1),oc.F_q_2d(:,ii),'-','Marker','.');
%     hold on;
% end
% xlabel('x');
% ylabel('y');
% zlabel('F_q_{2d}');
% %axis equal;

xles=interp1(w.y,w.xLE,oc.y); % leading edges
cs=sum(oc.dx,1); % chords

oc.xNorm=(oc.x-xles)./cs; % Normalised from 0 to 1 along chord
oc.yNorm=oc.y/w.s; % Normalised from 0 to 1 along span

oc.xNorm=sum(oc.xNorm,2)/size(oc.xNorm,2); % average for meshgrid

oc.xNormold=[0;oc.xNorm;1]; % saving arrays, will be used later
oc.yNormold=[0,oc.yNorm,1];

oc.xNorm=[0;oc.xNorm;1]; % adding 0 and 1 positions
oc.yNorm=[0,oc.yNorm,1]; % adding 0 and 1 positions

[oc.yNorm,oc.xNorm]=meshgrid(oc.yNorm,oc.xNorm);

temp=oc.F_q_2d;
oc.F_q_2d=zeros(size(oc.F_q_2d)+2); % adding the [0,1] points for spanwise positions and
    chordwise positions
oc.F_q_2d(2:end-1,2:end-1)=temp;
oc.F_q_2d(2:end-1,1)=(oc.F_q_2d(2:end-1,3)-oc.F_q_2d(2:end-1,2))./(oc.yNorm(1,3)-oc.yNorm(1,2))
    .* (0-oc.yNorm(1,2))+oc.F_q_2d(2:end-1,2);

figure;
for ii=1:Ns+2
    plot3(oc.xNormold,oc.yNormold(ii)*ones(Nc+2,1),oc.F_q_2d(:,ii),'-','Marker','.');
    hold on;
end
xlabel('x');
ylabel('y');
zlabel('F_q_{2d}');
%axis equal;

% figure;
% plot3(oc.x,oc.y,oc.F_q,'. ');
% xlabel('x');
% ylabel('y');

% % Not necessary, only to check
% oc.Cl_c=sum(oc.F_q,1)./sum(oc.dx,1)./oc.swidth;
% oc.Cl_cref=sum(oc.F_q,1)./oc.swidth/w.cbar;
%
% oc.Cl_c2=sum(oc.dCp.*oc.dx,1)./sum(oc.dx,1);
% oc.Cl_cref2=sum(oc.dCp.*oc.dx,1)./w.cbar
%
%
% figure;
% plot(oc.y,oc.Cl_c,'Marker','.');

```



```

% hold on;
% plot(os.y,os.cl_c,'Marker','.');
%
% figure;
% hold on;
% plot(oc.y,oc.Cl_cref,'Marker','.');
% hold on;
% plot(os.y,os.cl_cref,'Marker','.');

```

```
end
```

B.11 conv_controlSurfaceIn

```

function[]=conv_controlSurfaceIn(pfm,panels,controlin,fid,flipcsys,ss)
% CONV_CONTROLSURFACEIN - adds a new section to the .avl file corresponding
% to the start/end of the control surface. This is necessary because
% control surfaces will most likely not begin/end at the defined planform
% sections, but somewhere before/after.
%
% ATTENTION: this function must be called within conv_avlin or bwb_avlin.
% A.k.a., your current folder MUST BE .\avl-tree\avl.in. Otherwise, the
% code will fail.
%
% INPUTS (structs):
%   pfm - planform struct, eg. w for wing, h for hstab, v for vstab.
%   panels - panel spacing definition. Has fields:
%       Nspan
%       Sspace
%       doBool - if 1, will add Nspan and Sspace to the .avl file. if 0, will skip altogether
%   controlin - control line inputs. Has fields:
%       name
%       gain
%       Xhinge
%       XYZhvec
%       SgnDup
%   fid - ID to the current open file (.avl file being written)
%   flipcsys - 1 to switch y and z coords (useful for vstab), 0 to do normal.
%   ss - normalised y-position of section (y/s, so 0 at root, 1 at tip)
%
% EXAMPLE USAGE FOR WING CONTROL SUTFACES (COPY THIS CODE INTO
% conv_avlin.m OR bwb_avlin.m, IN THE RIGHT PLACE WHERE YOU
% WANT THE EXTRA SECTION):
% % % % % Pretend section
% % % % % panels.Nspan=8;
% % % % % panels.Sspace=1;
% % % % % panels.doBool=1;
% % % % % controlin.name='controlsurf';
% % % % % controlin.gain=1;
% % % % % controlin.Xhinge=0.8;
% % % % % controlin.XYZhvec=[0,1,0];
% % % % % controlin.SgnDup=1;
% % % % % sscontrol = 0.1;
% % % % %
% % % % % conv_controlSurfaceIn(w,panels,controlin,fid,0,sscontrol);

%% Computations
% Geometry interpolation
xLE=interp1(pfm.ss,pfm.xLE,ss,'linear');
y=ss*pfm.s;
z=interp1(pfm.ss,pfm.z,ss,'linear');

```

```

if flipcsys==1 % if a vertical stabiliser, switch the y and z coordinates
    intern=z;
    z=y;
    y=intern;
end

c=interp1(pfm.ss,pfm.c,ss,'linear');
a=interp1(pfm.ss,pfm.twist,ss,'linear');

% Airfoil coordinates
cd aerofoil_analysis_xfoil

if length(pfm.afname)>1 && iscell(pfm.afname) % if multiple foils applied to the surface
    [~,ni]=find(pfm.ss > ss, 1);

    nss=pfm.ss(ni-1)+(ss-pfm.ss(ni-1))/(pfm.ss(ni)-pfm.ss(ni-1)); % re-normalised ss. goes from
    0 to 1 for the section around the current ss. Don't worry about understanding it.

    fid1=fopen(pfm.afname{ni-1}, 'r+');
    afcoords1=fscanf(fid1, '%f', [2,inf]); % airfoil coordinates inboard of the desired section
    fclose(fid1);

    fid1=fopen(pfm.afname{ni}, 'r+');
    afcoords2=fscanf(fid1, '%f', [2,inf]); % airfoil coordinates outboard of the desired section
    fclose(fid1);

    [~,si1]=min(afcoords1(:,1)); % index corresponding to the LE
    [~,si2]=min(afcoords2(:,1)); % index corresponding to the LE

    top1=afcoords1(1:si1,:); % top and bottom surfaces for the 2 foils
    bot1=afcoords1(si1:end,:);
    top2=afcoords2(1:si2,:);
    bot2=afcoords2(si2:end,:);

    % Weighted average of the geometries. x/c values used from the inboard
    % airfoil.
    top=top1(:,2)*(1-nss)+interp1(top2(:,1),top2(:,2),top1(:,1))*nss;
    bot=bot1(:,2)*(1-nss)+interp1(bot2(:,1),bot2(:,2),bot1(:,1))*nss;

    afcoords=[top1(:,1),top;bot1(2:end,1),bot(2:end)];
else % if surface has 1 foil throughout
    fid1=fopen(pfm.afname, 'r+');
    afcoords=fscanf(fid1, '%f', [2,inf]);
    fclose(fid1);
end

% Plotting, trying, etc.
if 0
    xs=[0:0.0001:1]';
    figure;
    hold on;
    plot(afcoords1(:,1),afcoords1(:,2),'.b');
    plot(afcoords2(:,1),afcoords2(:,2),'.r');
    plot(afcoords(:,1),afcoords(:,2),'.k');
    plot(xs,interp1(top1(:,1),top1(:,2),xs,'pchip'),'-b');
    plot(xs,interp1(bot1(:,1),bot1(:,2),xs,'pchip'),'-b');
    plot(xs,interp1(top2(:,1),top2(:,2),xs,'pchip'),'-r');
    plot(xs,interp1(bot2(:,1),bot2(:,2),xs,'pchip'),'-r');
    plot(xs,interp1(top1(:,1),top,xs,'pchip'),'-k');
    plot(xs,interp1(bot1(:,1),bot,xs,'pchip'),'-k');
    legend('inboard','outboard','current');
end

```

```

axis equal;
end

cd ..

%% Printing
fprintf(fid,'# (Interpolated planform section for start/end of control surfaces))\n');
fprintf(fid,' SECTION ! (keyword)\n');
if panels.doBool==1
    fprintf(fid,' %.4f %.4f %.4f %.4f %.4f %.4f %.4f ! Xle Yle Zle
Chord Ainc [ Nspan Sspace ]\n',xLE,y,z,c,a,panels.Nspan,panels.Sspace);
else
    fprintf(fid,' %.4f %.4f %.4f %.4f %.4f ! Xle Yle Zle Chord Ainc\n',xLE,
y,z,c,a);
end
fprintf(fid,' AIRFOIL 0.0 1.0 ! (keyword) [ optional x/c range ]\n');
%fprintf(fid,' %s ! filename string\n',['D:\Documents\Imperial\
Y3\Subjects\96005.GDP\avl_analysis\avl_tree\'',w.afname1]);
fprintf(fid,' %f %f\n',afcoords');

if 1
% Control surface
fprintf(fid,' CONTROL ! (keyword)\n');
fprintf(fid,' %s %.4f %.4f %.4f %.4f %.4f ! name, gain, Xhinge,
XYZhvec, SgnDup\n',controlin.name,controlin.gain,controlin.Xhinge,controlin.XYZhvec(1),
controlin.XYZhvec(2),controlin.XYZhvec(3),controlin.SgnDup);
fprintf(fid,'#\n');
end
end

```

C Code - Blended Wing Body

Note the `planformcalcs` script were bundled into `buildup`; and `controlSurfaceIn` was not included, even though similar code was used by the control team. This is due to practicality and including only code that runs automatically and sequentially (without manually running AVL) in the appendices.

C.1 bwb_main

```
% Housekeeping
clear;
clc;
close all;

% Path for cmd:
% D:
% cd Documents/Imperial/Y3/Subjects/"96005-GDP"/avl_analysis/avl_tree
% .\avl.exe
% load .\avl_in\bwb.avl

%% Booleans
bool.plt=1; % plotting
bool.dhdrldfudge=1; % due to dihedral, the lift and drag at the root spike up.
                    % if this is 1, rootmost point is neglected when
                    % interpolating.
bool.chordwisedistrib=1; % 1 to also find the lift distributions chordwise
bool.thicknesscorrection=[1,1,1,1]; % whether or not to correct lift for airfoil thickness, for
    each wing section
bool.avldebug=1; % whether or not to show text output from avl
bool.fullanalysis=0; % if 0, only performs analysis to give desired cl. if 1, finds everything
    including lift-curve slopes, etc. and stall AoA
bool.printoutinfo=1; % 1 to print out cd, oswals, etc. (useful info for wing design)

%% Inputs - parametrisation
% Wing divided into 3 sections, 4 spanwise positions. These are root (0),
% mid1 (1), mid2 (2), tip (3)

% Wing
w.pos=[0,0,0]; % position of root LE in x,y,z space (relative to global origin)
w.S=1110.04; % wing area
w.AR=4.9; % aspect ratio

w.ss=[0.2712,0.3390]; % spanwise positions of sections 1 and 2, as % of semispan
w.tpr=[0.59952,0.4360,0.3211]; % taper for each section, c_i/c_{i-1}
%w.tpr=[0.5995000000000000    0.6060000000000000    0.1320000000000000]
w.swp=[30.22,56,40]; % 1/4 chord sweep for each section
w.dhtrl=[0,0,0];

% Angles
w.i0=0; % wing incidence, deg
w.twist=[0,0,-1,-2.1]; % wing twist for positions 0,1,2. Note twist at position 1 should always
    be zero.

% Airfoils
w.afname={'MH93.dat','MH93.dat','rae2822.dat','rae2822.dat'};

% Flight conditions
fc.M=0.8;
fc.alt=35000/3.281; % in metres

% Runtime parameters
rn.Cl=0.1552; % Desired Cl
```

```

% CG position
wb.xcg=0.1; % wrt global CSYS
wb.zcg=0;

% Winglet
wlet.do=0; % boolean to add or not
wlet.A1=16.65; % area of one winglet
wlet.tpr=0.6;
wlet.swp=60;
wlet.cant=80;
wlet.span=5.904;
wlet.twist=w.twist(end);
wlet.Nspan=3;
wlet.Sspace=0.7;

%% AVL inputs
in.Nchord=20; % number of chordwise panels
in.Cspace=1.0; % chordwise spacing parameter
in.Nspan=[12,4,23];
in.Sspace=[1,1,1]*0.7;

%   parameter                      spacing
%   -----
%
%   3.0      equal      |  |  |  |  |  |  |  |  |
%
%   2.0      sine       || |  |  |  |  |  |  |  |
%
%   1.0      cosine     || |  |  |  |  |  |  |  ||
%
%   0.0      equal      |  |  |  |  |  |  |  |  |
%
%  -1.0      cosine     || |  |  |  |  |  |  |  ||
%
%  -2.0     -sine       |  |  |  |  |  |  |  |  ||
%
%  -3.0      equal      |  |  |  |  |  |  |  |  |
%
% An intermediate parameter value will result in a blended distribution.

%% Running
[w,ot,os,oc]=bwb_multipleAnalyses(bool,w,fc,wb,rn,in,wlet);

```

C.2 bwb_multipleAnalyses

```

function[w,ot,os,oc]=bwb_multipleAnalyses(bool,w,fc,wb,rn,in,wlet)
% BWB_MULTIPLEANALYSES - sets up and runs conv.buildup for the analyses
% requested.
% INPUTS: every input struct.
%   bool - booleans
%   w - wing
%   fc - flight conditions
%   wb - weight & balance
%   rn - misc. avl run parameters
%   in - misc. avl input parameters
%
% OUTPUTS:
%   w - wing (updated)
%   ot - outputs for the whole aircraft

```

```

%      os - spanwise distributions
%      oc - chordwise distributions

%% Saving backup variables
if bool.fullanalysis==1 % Saving backups if full analysis is requested (some variables change
    and add/remove fields when analysis is run)
    back.w=w;
    back.rn=rn;

    animateBisetion=0; % feel free to change this. 1 for animating the bisection method, 0 for
        not.
end

%% Running the main analysis
[w,ot,os,oc]=bwb_buildup(bool,w,fc,wb,rn,in,wlet); % runs the main analysis

%% Running full analyses
if bool.fullanalysis==1 % If full analysis is requested
    %% Setup
    % Changes bools to avoid unnecessary plotting / printing of values
    bool.chordwisedistrib=0;
    bool.plt=0;
    bool.wingonly=1;
    bool.avldebug=0;
    bool.printoutinfo=0;

    % AoAs
    aoas=[1,4]; % 2 aoas for calculating lift curve slopes

    %% Analysis case 1 - first AoA
    clear('rn');
    rn.aoa=aoas(1);
    [~,ot1,os1,~]=bwb_buildup(bool,back.w,fc,wb,rn,in,wlet); % runs the main analysis

    %% Analysis case 2 - second AoA
    rn.aoa=aoas(2);
    [~,ot2,os2,~]=bwb_buildup(bool,back.w,fc,wb,rn,in,wlet); % runs the main analysis

    %% Calculation of wing parameters
    ot.aw=(ot2.Cl-ot1.Cl)/((ot2.alpha-ot1.alpha)*pi/180); % overall wing lift curve slope in
        rad^-1
    c=ot1.Cl-ot.aw*(-ot1.Cl)*pi/180;
    ot.alpha0w=-c/ot.aw; % overall wing zero-lift aoa, in rad
    os.alocal=(os2.Cl-c-os1.Cl-c)/((ot2.alpha-ot1.alpha)*pi/180); % local lift curve slope
    os.a0local=(ot1.alpha*pi/180-os1.Cl-c./os.alocal)*180/pi;

    %% STALL - spanwise Clmax distribution
    % We use xfoil data and a linear blend to find the spanwise Clmax
    % distribution.

    sss=0:0.001:1; % finer spanwise station array

    [~,i1]=find(sss>=w.ss(2),1); % index for the mid1 section of planform
    [~,i2]=find(sss>=w.ss(3),1); % index for the mid2 section of planform

    cs=interp1(w.ss,w.c,sss,'linear'); % chords
    [T,~,~,rho]=atmosisa(fc.alt);
    mu=1.716e-5*(T/273)^1.5*(273+111)/(T+111); % viscosity calculation
    Res=rho*fc.M*sqrt(1.4*287*T)*cs/mu; % reynolds along span

    cd aerofoil_analysis_xfoil

```


C.3 bw_b buildup

```
function[w,ot,os,oc]=bw_b.buildup(bool,w,fc,wb,rn,in,wlet)
% BWB_BUILDUP - main assembly script that runs functions sequentially.
% INPUTS: every input struct.
%     bool - booleans
%     w - wing
%     fc - flight conditions
%     wb - weight & balance
%     rn - misc. avl run parameters
%     in - misc. avl input parameters
%
% OUTPUTS:
%     w - wing (updated)
%     ot - outputs for the whole aircraft
%     os - spanwise distributions
%     oc - chordwise distributions

%% Deletion of files
[~,~]=dos('del .\avl\tree\avl\in\bw_b.avl'); % input files
[~,~]=dos('del .\avl\tree\avl\in\bw_b.run');
[~,~]=dos('del .\avl\tree\avl\out\bw_b.fs'); % output files
[~,~]=dos('del .\avl\tree\avl\out\bw_b.ft');
[~,~]=dos('del .\avl\tree\avl\out\bw_b.fe');

%% Planform calculations
w.b=sqrt(w.AR*w.S); % span
w.s=w.b/2; % semispan
w.ss=[0,w.ss,1]; % spanwise stations for sections
w.y=w.ss*w.s; % y-position of each section

% Finding root chord
sbar=w.y(2:end)-w.y(1:end-1);
k=sbar.*(1+w.tpr).*cumprod([1,w.tpr(1:end-1)]);
w.c0=w.S/sum(k);
w.c=w.c0*cumprod([1,w.tpr]);

if wlet.do==1
    w.b=w.b+2*wlet.span*cosd(wlet.cant);
    w.s=w.b/2;
    w.S=w.S+wlet.A1*2;
    w.y=[w.y,w.y(end)+wlet.span*cosd(wlet.cant)];
    sbar=w.y(2:end)-w.y(1:end-1);
    w.AR=w.b^2/w.S;
    w.ss=w.y/w.y(end);
    w.tpr=[w.tpr,wlet.tpr];
    w.c=[w.c,w.c(end)*wlet.tpr];
    w.swp=[w.swp,wlet.swp];
    w.dhtrl=[w.dhtrl,wlet.cant];
    w.twist=[w.twist,wlet.twist];
    w.afname=[w.afname,w.afname{end}];
    in.Nspan=[in.Nspan,wlet.Nspan];
    in.Sspace=[in.Sspace,wlet.Sspace];
end

% Sweeps
w.swpLE=atan((sbar.*tand(w.swp)+(1/4-0)*(w.c(1:end-1)-w.c(2:end)))/sbar)*180/pi;
w.swpTE=atan((sbar.*tand(w.swp)+(1/4-1)*(w.c(1:end-1)-w.c(2:end)))/sbar)*180/pi;

w.xLE=[0,sbar.*tand(w.swpLE)];
w.xLE=cumsum(w.xLE);
```

```

w.xTE=[w.c0,sbar.*tand(w.swpTE)];
w.xTE=cumsum(w.xTE);
w.xTE=w.xLE+w.c;

w.z=[0,sbar.*sind(w.dhdr1)];
w.z=cumsum(w.z);

if wlet.do==1
    w.z(end)=w.z(end-1)+wlet.span*sind(wlet.cant);
end

w.i=w.i0+w.twist; % incidence of each seciton

% MAC
a=(w.c(2:end)-w.c(1:end-1))./(w.y(2:end)-w.y(1:end-1));
b=w.c(1:end-1)-a.*w.y(1:end-1);
w.cbar=(2/w.S)*sum(a.^2.*(w.y(2:end).^3-w.y(1:end-1).^3)/3+a.*b.*(w.y(2:end).^2-w.y(1:end-1).^2)+b.^2.*(w.y(2:end)-w.y(1:end-1)));
w.ybar=interp1(w.c,w.y,w.cbar,'linear');
w.xLEbar=interp1(w.c,w.xLE,w.cbar,'linear');

%% Plotting Planform
if bool.plt==1
    bwb_planformplot(w,in,wlet);
end

%% Generating the AVL files
bwb_avlin(w,fc,wb,in,bool,wlet); % Generates .avl input file
bwb_avlrun(rn,fc,bool); % Generates .run analysis file

%% Running AVL
cd avl_tree
if bool.avldebug==1
    dos('.\avl.exe < .\avl.in\bwb.run'); % runs avl, outputting text to command window
else
    [~,~]=dos('.\avl.exe < .\avl.in\bwb.run'); % runs avl without any text output
end
cd ..

%% Output Processing
[ot,os]=bwb_avlout(rn,w); % Grabs the output data from text files
[ot,os]=bwb_outproc(ot,os,bool,w,fc,wlet); % Processes the output data

if bool.chordwisedistrib==1 % chordwise distributions of lift
    oc=bwb_chordwisedistrib(in,w,os,ot);
else
    oc=NaN;
end
end

```

C.4 bwb_planformplot

```

function []=bwb_planformplot(w,in,wlet)
% BWB_PLANFORMPLOT - Plots the planform on the left (with panels), and
% zooms in on the kinks on the right (to check the panel spacing at the kinks)
%
% INPUTS (structs):
%     w - wing
%     in - avl input file
%     bool - boolean

```

```

figure('Position',[ (1920-1200)/2, (1080-600)/2, 1200, 600]); % creates figure

for iii=2:4 % plots the full planform on both sides of the figure
    switch iii
        case 2
            subplot(2,2,[1,3]);
        case 3
            subplot(2,2,2);
        case 4
            subplot(2,2,4);
    end

    hold on;

    plot(w.pos(1)+w.xLE,w.y,'-k'); % LE
    plot(w.pos(1)+w.xTE,w.y,'-k','HandleVisibility','off'); % TE
    plot(w.pos(1)+w.xLE+w.c/4,w.y,':k'); % 1/4 chord

    for ii=1:length(w.y)
        plot(w.pos(1)+[w.xLE(ii),w.xTE(ii)],w.y(ii)*[1,1],'-k','HandleVisibility','off'); %
            chords
    end

    plot(w.pos(1)+w.xLEbar+[0,w.cbar],w.ybar*[1,1],'--k'); % MAC

    % Panels
    ssp=[spacing(in.Nspan(1),in.Sspace(1))*w.ss(2),...
        w.ss(2)+spacing(in.Nspan(2),in.Sspace(2))*(w.ss(3)-w.ss(2)),...
        w.ss(3)+spacing(in.Nspan(3),in.Sspace(3))*(w.ss(4)-w.ss(3))]; % spanwise panel spacing

    if wlet.do==1
        ssp=[ssp,w.ss(4)+spacing(in.Nspan(4),in.Sspace(4))*(w.ss(5)-w.ss(4))];
    end

    xlep=interp1(w.ss,w.xLE,ssp,'linear'); % interpolating leading edge positions
    cp=interp1(w.ss,w.c,ssp,'linear'); % interpolating chords

    for ii=1:length(cp) % for each panel edge
        plot(w.pos(1)+xlep(ii)+[0,cp(ii)],ssp(ii)*w.s*[1,1],':b'); % plots panel edge (chord)
    end

    xlabel('x (m)');
    ylabel('y (m)');

    switch iii % Titles
        case 2
            title('Planform');
            legend('planform','1/4 chord','MAC','Location','northwest');
            axis equal;
        case 3
            title('Kink 1 detail');
            xlim(w.xLE(2)+[0,w.c(2)]+0.1*w.c(2)*[-1,1]); % zooms into kink
            ylim(w.y(2)+[-1,1]*w.s*0.05);
        case 4
            title('Kink 2 detail');
            xlim(w.xLE(3)+[0,w.c(3)]+0.1*w.c(3)*[-1,1]); % zooms into kink
            ylim(w.y(3)+[-1,1]*w.s*0.05);
    end
end
end

```

C.5 bw_b_avlin

```

function[]=bwb_avlin(w,fc,wb,in,bool,wlet)
% BWB.AVLIN - generates AVL input file for bwb aircraft
%
% INPUTS: a series of structs
%     w - wing
%     fc - flight conditions
%     wb - weight and balance
%     in - miscellaneous avl inputs for the .avl file
%     h - horizontal stabiliser
%     v - vertical stabiliser
%     bool - booleans (whether or not to plot, whether or not to include
%           tail in analysis, etc)
%
% OUTPUTS: none

%% Grabbing airfoil coordinates from files
cd aerofoilanalysis_xfoil % changes to airfoil directory

% Wing, root
fid=fopen(w.afname{1},'r+');
w.afCoords0=fscanf(fid,'%f',[2,inf]); % takes coordinates from file and stores in an array
fclose(fid);

% Wing, mid 1
fid=fopen(w.afname{2},'r+');
w.afCoords1=fscanf(fid,'%f',[2,inf]);
fclose(fid);

% Wing, mid 2
fid=fopen(w.afname{3},'r+');
w.afCoords2=fscanf(fid,'%f',[2,inf]);
fclose(fid);

% Wing, tip
fid=fopen(w.afname{4},'r+');
w.afCoords3=fscanf(fid,'%f',[2,inf]);
fclose(fid);

cd ..

%% Setup
fid=fopen('bwb.avl','w+'); % creates the input file

fprintf(fid,'GDP.BlendedWingBody\n');
fprintf(fid,'%4f                                ! Mach\n',fc.M); % mach #
fprintf(fid,'0      0      0.0                ! iYsym iZsym Zsym\n'); % defining planes of
symmetry
fprintf(fid,'%4f    %4f    %4f                ! Sref   Cref   Bref\n',w.S,w.cbar,w.b); %
reference values
fprintf(fid,'%4f    %4f    %4f                ! Xref   Yref   Zref\n',wb.xcg,0,wb.zcg); % cg
position
fprintf(fid,'0.020                            ! CDp   (optional)\n'); % default profile drag
coefficient added to geometry

fprintf(fid,'#\n');
fprintf(fid,'#-----\n');
fprintf(fid,'#\n');

%% Wing
fprintf(fid,'SURFACE                        ! (keyword)\n');
fprintf(fid,'      Wing                    ! surface name string\n');
fprintf(fid,'      %d    %4f                ! Nchord  Cspace\n',in.Nchord,in.Cspace); %
chordwise panel definition

```

```

fprintf(fid,'#\n');
fprintf(fid,'      SCALE                                ! (keyword)\n'); % sets scale to 1
fprintf(fid,'      1.0      1.0      1.0                ! Xscale Yscale Zscale\n');
fprintf(fid,'#\n');
fprintf(fid,'      TRANSLATE                                ! (keyword)\n'); % moves wing to respective
      position
fprintf(fid,'      %.4f      %.4f      %.4f                ! dX dY dZ\n',w.pos(1),w.pos(2),w.pos(3));
fprintf(fid,'#\n');
fprintf(fid,'      ANGLE                                ! (keyword)\n'); % sets wing incidence
fprintf(fid,'      %.4f                                ! dAinc\n',w.i0);
fprintf(fid,'#\n');
fprintf(fid,'      YDUPLICATE                                ! (keyword)\n'); % generates the other side of
      the wing
fprintf(fid,'      0.0                                ! Ydupl\n');
fprintf(fid,'#\n');
fprintf(fid,'#\n');

ii=0;

% Root section
ii=ii+1;
fprintf(fid,'#      (root section)\n');
fprintf(fid,'      SECTION                                ! (keyword)\n');
fprintf(fid,'      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      ! Xle Yle Zle Chord
      Ainc [ Nspan Sspace ]\n',w.xLE(ii),w.y(ii),w.z(ii),w.c(ii),w.twist(ii),in.Nspan(ii),
      in.Sspace(ii)); % geometry
fprintf(fid,'      AIRFOIL      0.0      1.0                ! (keyword) [ optional x/c range ]\n'); %
      airfoil
fprintf(fid,'      %f      %f\n',w.afCoords0');
fprintf(fid,'#\n');

% Mid 1 section
ii=ii+1;
fprintf(fid,'#      (mid1 section)\n');
fprintf(fid,'      SECTION                                ! (keyword)\n');
fprintf(fid,'      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      ! Xle Yle Zle Chord
      Ainc [ Nspan Sspace ]\n',w.xLE(ii),w.y(ii),w.z(ii),w.c(ii),w.twist(ii),in.Nspan(ii),
      in.Sspace(ii)); % geometry
fprintf(fid,'      AIRFOIL      0.0      1.0                ! (keyword) [ optional x/c range ]\n'); %
      airfoil
fprintf(fid,'      %f      %f\n',w.afCoords1');
fprintf(fid,'#\n');

% Mid 2 section
ii=ii+1;
fprintf(fid,'#      (mid2 section)\n');
fprintf(fid,'      SECTION                                ! (keyword)\n');
fprintf(fid,'      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      ! Xle Yle Zle Chord
      Ainc [ Nspan Sspace ]\n',w.xLE(ii),w.y(ii),w.z(ii),w.c(ii),w.twist(ii),in.Nspan(ii),
      in.Sspace(ii)); % geometry
fprintf(fid,'      AIRFOIL      0.0      1.0                ! (keyword) [ optional x/c range ]\n'); %
      airfoil
fprintf(fid,'      %f      %f\n',w.afCoords2');
fprintf(fid,'#\n');

% Tip section
ii=ii+1;
fprintf(fid,'#      (tip section)\n');
fprintf(fid,'      SECTION                                ! (keyword)\n');
if wlet.do==1
    fprintf(fid,'      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      %.4f      ! Xle Yle Zle
      Chord Ainc [ Nspan Sspace ]\n',w.xLE(ii),w.y(ii),w.z(ii),w.c(ii),w.twist(ii),in.Nspan
      (ii),in.Sspace(ii)); % geometry

```

```

else
    fprintf(fid, '          %.4f   %.4f   %.4f   %.4f   %.4f   ! Xle Yle Zle   Chord Ainc\n',
            w.xLE(ii),w.y(ii),w.z(ii),w.c(ii),w.twist(ii)); % geometry
end
fprintf(fid, '          AIRFOIL          0.0   1.0          ! (keyword)   [ optional x/c range ]\n'); %
    airfoil
fprintf(fid, '          %f   %f\n',w.afCoords3');
fprintf(fid, '#\n');

if wlet.do==1
    % Winglet section
    ii=ii+1;
    fprintf(fid, '#   (wlet section)\n');
    fprintf(fid, '   SECTION                      ! (keyword)\n');
    fprintf(fid, '          %.4f   %.4f   %.4f   %.4f   %.4f   ! Xle Yle Zle   Chord Ainc\n',
            w.xLE(ii),w.y(ii),w.z(ii),w.c(ii),w.twist(ii)); % geometry
    fprintf(fid, '          AIRFOIL          0.0   1.0          ! (keyword)   [ optional x/c range ]\n');
        % airfoil
    fprintf(fid, '          %f   %f\n',w.afCoords3');
    fprintf(fid, '#\n');
end
% Can add sections for flaps, ailerons, etc.

fprintf(fid, '#\n');
fprintf(fid, '#-----\n');
fprintf(fid, '#\n');

fclose(fid); % closes file

%% Moving file to appropriate folder
copyfile('bwb.avl','avl_tree/avl.in');
delete('bwb.avl');
end

```

C.6 bwb_avlrun

```

function[]=bwb_avlrun(rn,fc,bool)
% BWB_AVLRUN - generates AVL run file for bwb aircraft
%
% INPUTS - structs:
%   rn - misc AVL inputs for the .run file
%   fc - flight conditions
%   bool - booleans, for eg. for plotting or not

%% Setup
cd avl_tree/avl.in % moves to input folder
fid=fopen('bwb.run','w+'); % creates the input file

% Air density
[~,~,~,rho]=atmosisa(fc.alt);

%% Writing
fprintf(fid, 'LOAD %s \n','.\avl.in\bwb.avl'); % loads geometry from .avl file
% fprintf(fid, 'OPER \n');
% fprintf(fid, 'G \n');
% fprintf(fid, 'CA \n');
fprintf(fid, 'OPER \n');

if isfield(rn,'Cl') % if we are running a specific cl analysis
    fprintf(fid, 'Cl \n');
    fprintf(fid, 'C %.4f \n',rn.Cl); % target CL. Will run an analysis where AoA changes
end

```

```

        until desired Cl is reached
    fprintf(fid, 'D %.4f \n', rho); % density
    fprintf(fid, '\n');
elseif isfield(rn, 'aoa') % if we are running a specific aoa analysis
    fprintf(fid, 'A A %.4f \n', rn.aoa); % target CL. Will run an analysis where AoA changes
        until desired Cl is reached
end

fprintf(fid, 'X \n'); % runs the analysis

% Save File for Total Force
fprintf(fid, 'FT\n');
fprintf(fid, '%s \n', '.\avl.out\bwb.ft');
%fprintf(fid, 'O \n');
%fprintf(fid, '\n');

% Save File for Strip Force (spanwise lift/drag distributions)
fprintf(fid, 'FS\n');
fprintf(fid, '%s \n', '.\avl.out\bwb.fs');
%fprintf(fid, 'O \n');
%fprintf(fid, '\n');

if bool.chordwisedistrib==1
    % Save File for Element Force (spanwise and chordwise lift distribution)
    fprintf(fid, 'FE\n');
    fprintf(fid, '%s \n', '.\avl.out\bwb.fe');
    %fprintf(fid, 'O \n');
    %fprintf(fid, '\n');
end

fprintf(fid, '\n');
fclose(fid); % closes file
cd ..
cd ..
end

```

C.7 bwb_avlout

```

function[ot,os]=bw_b_avlout(rn,w)
% BWB_AVLOUT - gets output from avl analysis output files
%
% INPUTS (structs)
%     rn - miscellaneous avl inputs for the .run file
%     w - wing

%% Copy the files to current folder
%cd avl_tree\avl.out % moves to respective folder
copyfile('avl_tree/avl.out/bwb.fs', 'bwb.fs');
copyfile('avl_tree/avl.out/bwb.ft', 'bwb.ft');
% if bool.chordwisedistrib==1
%     copyfile('avl_tree/avl.out/conv.fe', 'conv.fe');
% end

%% Reading Total forces
fid=fopen('bwb.ft', 'r+');

ot.alpha=myAvlFileRead(fid, 'Alpha'); % AoA

```

```

ot.Cl =myAvlFileRead(fid,'CLtot'); % Cl
ot.Cdi =myAvlFileRead(fid,'CDff'); % lift=induced Cd
ot.e =myAvlFileRead(fid,'e'); % oswaldis

fclose(fid);

%% Reading Strip forces
fid=fopen('bwb.fs','r+');

myAvlFileRead(fid,'cm_c/4');
data=fscanf(fid,'%f',[13,inf]); % scans full data table
fclose(fid);

% Processing of strip forces
os.y=data(2,:); % y-positions
os.c=data(3,:); % chords
ccl=data(5,:); % cl * section chord
os.cl_cref=ccl/w.cbar; % cl, using MAC as reference chord
os.cl_c=ccl./os.c; % cl, using section chord as reference chord
os.alphai=-data(6,:); % induced AoAs
os.cdi=data(9,:); % induced Cd
%os.cm14=data(11,:);

%% Delete the temporary files in this folder
delete('bwb.fs');
delete('bwb.ft');
% if bool.chordwisedistrib==1
%     delete('conv.fe');
% end
% cd ..
% cd ..
end

```

C.8 bwb_outproc

```

function[ot,os]=bwb_outproc(ot,os,bool,w,fc,wlet)
% BWB_OUTPROC - processes the output, estimates viscous drag, etc
%
% INPUTS - structs:
%     ot - total aero data (cl, cdi, e)
%     os - spanwise aero distributions
%     bool - boolean struct
%     w - wing
%     fc - flight conditions
%
% OUTPUTS - structs
%     ot - total aero data (cl, cdi, e)
%     os - spanwise aero distributions

%% Processing and smooth spanwise distributions
fi=bool.dhdr1fudge+1; % 1 if no fudge, 2 if fudge (skips rootmost point in distribution)

os.ss=os.y/w.s; % spanwise stations - 0 at root, 1 at tip
sss=0:0.001:1; % finer spanwise station array
cls=interp1(os.ss(fi:end),os.cl_cref(fi:end),sss,'pchip'); % interpolating cl
cdi=interp1(os.ss(fi:end),os.cdi(fi:end),sss,'pchip'); % interpolating cdi

[~,i1]=find(sss>=w.ss(2),1); % index for the mid1 section of planform
[~,i2]=find(sss>=w.ss(3),1); % index for the mid1 section of planform

```



```

is=interp1(w.ss,w.i,sss,'linear'); % incidences along span
as=is+ot.alpha+interp1(os.ss(fi:end),os.alphai(fi:end),sss,'pchip'); % "angles of attack"

%% Viscous drag estimation
cs=interp1(w.ss,w.c,sss,'linear');

[T,~,~,rho]=atmosisa(fc.alt);
mu=1.716e-5*(T/273)^1.5*(273+111)/(T+111); % viscosity calculation
Res=rho*fc.M*sqrt(1.4*287*T)*cs/mu; % reynolds

cd aerofoilanalysis_xfoil
load(w.afname{1}(1:end-4)); % Grabs aero data for root airfoil
xfoil0=thisfoil;
load(w.afname{2}(1:end-4)); % Grabs aero data for mid1 airfoil
xfoil1=thisfoil;
load(w.afname{3}(1:end-4)); % Grabs aero data for mid2 airfoil
xfoil2=thisfoil;
load(w.afname{4}(1:end-4)); % Grabs aero data for tip airfoil
xfoil3=thisfoil;

if wlet.do==1 % if also analysing the winglet, add its tip airfoil
    [~,i3]=find(sss>=w.ss(4),1); % index for the wlet root section of planform
    xfoil4=thisfoil;
    cd4=interp2(xfoil4.re,xfoil4.a,xfoil4.cd,Res,as); % wlet end
end

cd ..

% Blending airfoil data
cd0=interp2(xfoil0.re,xfoil0.a,xfoil0.cd,Res,as); % visc. drag for root airfoil
cd1=interp2(xfoil1.re,xfoil1.a,xfoil1.cd,Res,as); % mid1
cd2=interp2(xfoil2.re,xfoil2.a,xfoil2.cd,Res,as); % mid2
cd3=interp2(xfoil3.re,xfoil3.a,xfoil3.cd,Res,as); % tip

cdv=(cd0(1:i1).*(sss(i1)-sss(1:i1))+cd1(1:i1).*sss(1:i1))/sss(i1); % blending for inner
    section1
cdv=[cdv,(cd1(i1+1:i2).*(sss(i2)-sss(i1+1:i2))+cd2(i1+1:i2).*(sss(i1+1:i2)-sss(i1)))/(sss(i2)-
    sss(i1))]; % and inner section 2
cdv=[cdv,(cd2(i2+1:end).*(1-sss(i2+1:end))+cd3(i2+1:end).*(sss(i2+1:end)-sss(i2)))/(1-sss(i2))
    ]; % and outer section

if wlet.do==1 % if winglet also present, re-blend for the wing with that additional section
    cdv=(cd0(1:i1).*(sss(i1)-sss(1:i1))+cd1(1:i1).*sss(1:i1))/sss(i1); % blending for inner
        section1
    cdv=[cdv,(cd1(i1+1:i2).*(sss(i2)-sss(i1+1:i2))+cd2(i1+1:i2).*(sss(i1+1:i2)-sss(i1)))/(sss(
        i2)-sss(i1))]; % and inner section 2
    cdv=[cdv,(cd2(i2+1:i3).*(sss(i3)-sss(i2+1:i3))+cd3(i2+1:i3).*(sss(i2+1:i3)-sss(i2)))/(sss(
        i3)-sss(i2))]; % and outer section
    cdv=[cdv,(cd3(i3+1:end).*(1-sss(i3+1:end))+cd4(i3+1:end).*(sss(i3+1:end)-sss(i3)))/(1-sss(
        i3))]; % and winglet
end

% figure; % this figure shows how the blending along the span is happening
% hold on;
% plot(sss,cd0,'-b');
% plot(sss,cd1,'-r');
% plot(sss,cd2,'-g');
% plot(sss,cd3,'-m');
% if wlet.do==1
%     plot(sss,cd4,'-y');

```

```

% end
% plot(sss,cdv,'-k'); %%%%%%%%%%%%%%%
% plot(w.ss(2:3),[0,0],'xk');
% legend('root','mid1','mid2','tip','blended');
% if wlet.do==1
%     plot(w.ss(4),0,'xk');
% legend('root','mid1','mid2','tip','winglet tip','blended');
% end
% xlabel('normalised spanwise position');
% ylabel('cd_{visc}');
% title('Blending of airfoil sections spanwise');

%% Elliptical estimation - elliptical Cl that gives the same total lift
eldis=sqrt(1-sss.^2);
cls_ellip=eldis*trapz(sss,cls)/trapz(sss,eldis);
%cls_ellip=eldis*(w.S*ot.Cl/w.s/w.cbar/2)/trapz(sss,eldis);

%% Final computations
% Here, we will define Kl Kd Kdi:
% D = 0.5 rho V^2 S Cd
% Kd = S Cd = spanwise integral of c Cd

% % Ignore this block - it was used for testing/validation purposes
% Kl=trapz(sss*w.s,cls)*w.cbar % from our fine, interpolated distribution
% Kl=trapz(sss*w.s,cls_ellip)*w.cbar % from elliptical
% Kl=trapz(os.y,os.cl.c.*os.c) % from coarse, avl-outputted cl distribution (normalised with
%     sectional chord)
% Kl=trapz(os.y,os.cl.cref)*w.cbar % from coarse, avl-outputted cl distribution (normalised
%     with MAC)
% Kl=ot.Cl*w.S/2 % From total Cl required
%
% Kdi=trapz(sss*w.s,cdi.*cs) % from our fine, interpolated distribution
% Kdi=ot.Cdi*w.S/2 % from our coarse, avl-outputted cdi distribution
% Note that these 2 do not match. This is because the bottom one uses total
% Cdi from trafftz plane (which is more accurate). The top one
% (distribution) uses Cdi directly from the VLM. So everything is fine

Kdv=trapz(sss*w.s,cdv.*cs)*2; % integrates viscous drag along the wing
ot.Cdv=Kdv/w.S; % total viscous drag
ot.Cd=ot.Cdi+ot.Cdv; % total total drag

% Recall, of course, that
% Cl_c = lift / q c
% Cl_Cref = lift / q cref
%
% L = q S Cl = q cbar int(Cl_cref)`
%     = q int (Cl_c * c)

%% Printing out results
if bool.printoutinfo==1
    disp(['AoA = ',num2str(ot.alpha),' (angle of attack)']);
    %disp(['Cl = ',num2str(ot.Cl),' (lift)']);
    disp(['e = ',num2str(ot.e),' (oswalds efficiency)']);
    %disp(['Cdv = ',num2str(ot.Cdv),' (viscous drag)']);
    %disp(['Cdi = ',num2str(ot.Cdi),' (induced drag)']);
    disp(['Cd = ',num2str(ot.Cd),' (total drag)']);
    disp(['Cdv was ',num2str(ot.Cdv/ot.Cd*100),'% of Cd']);
    disp(['Cdi was ',num2str(ot.Cdi/ot.Cd*100),'% of Cd']);
    disp(['L/D = ',num2str(ot.Cl/ot.Cd),' (for wing only)']);
end

%% Plotting
if bool.plt==1

```

```

figure;
subplot(1,2,1); % Cl-span graph
hold on;
plot(os.ss,os.cl_cref,'xk','HandleVisibility','off'); % AVL data
plot(sss,cls,'-k'); % smoothed
plot(sss,cls_ellip,'-b'); % elliptical
xlabel('y/s');
ylabel('Cl');
legend('avl','elliptical');
%plot(os.ss,os.cl_cref(1)*sqrt(1-os.ss.^2));

subplot(1,2,2); % Cd-span graph (not very useful or accurate)
hold on;
plot(sss,cdv+cdi,'-k'); % total drag
plot(sss,cdv,'-b'); % viscous
plot(os.ss,os.cdi,'xr','HandleVisibility','off'); % lift-induced
plot(sss,cdi,'-r');
%plot(os.ss,os.cl_cref.*sind(os.alphai),'-g')
%plot(os.ss,os.cl_c.*sind(os.alphai),'-m')
xlabel('y/s');
ylabel('Cd');
legend('total','viscous','lift ind');
sgtitle('Spanwise aero');

end
end

```

C.9 bw_b_chordwisedistrib

```

function[oc]=bw_b_chordwisedistrib(in,w,os,ot)
% BWB_CHORDWISEDISTRIB - finds the chordwise (and spanwise) distribution
% of lift
%
% INPUTS (structs):
%   in - avl misc. inputs
%   w - wing
%   os and ot (not needed, but are used in some commented-out sections.
%
% OUTPUT:
%   oc - chordwise distribution struct

%% Initial thoughts
Nc=in.Nchord;
Ns=sum(in.Nspan);

% variable initialisations
oc.x=zeros(Nc,Ns);
oc.y=zeros(1,Ns);
oc.dCp=oc.x;
oc.dx=oc.x;
oc.swidth=oc.y; % strip width

%% Reading output file

copyfile('avl_tree/avl_out/bwb.fe','bwb.fe');
fid=fopen('bwb.fe','r+');
myAvlFileRead(fid,'
*****');

for ii=1:Ns
    oc.swidth(ii)=myAvlFileRead(fid,'Width'); % strip width
    myAvlFileRead(fid,'DX');

```

```

    data=fscanf(fid,'%f',[7,inf]); % full data stable for this strip
    oc.x(:,ii)=data(:,2);
    oc.y(ii)=data(1,3);
    oc.dx(:,ii)=data(:,5);
    oc.dCp(:,ii)=data(:,7);
end

fclose(fid);
delete('bwb.fe');
% cd ..
% cd ..

%% Calculations - remember spanwise goes horizontally, chordwise goes down

%oc.F_q=oc.dCp.*oc.dx.*oc.swidth; % this sums to give total Kl this is essentially force (lift)
    / dynamic head
oc.F_q_2d=oc.dCp.*oc.dx; % Basically, force distribution.

% figure; % Plots F_q_2d to show
% for ii=1:Ns
%     plot3(oc.x(:,ii),oc.y(ii)*ones(Nc,1),oc.F_q_2d(:,ii),'-','Marker','.');
%     hold on;
% end
% xlabel('x');
% ylabel('y');
% zlabel('F_q_{2d}');
% %axis equal;

xles=interp1(w.y,w.xLE,oc.y); % leading edges
cs=sum(oc.dx,1); % chords

oc.xNorm=(oc.x-xles)./cs; % Normalised from 0 to 1 along chord
oc.yNorm=oc.y/w.s; % Normalised from 0 to 1 along span

oc.xNorm=sum(oc.xNorm,2)/size(oc.xNorm,2); % average for meshgrid

oc.xNormold=[0;oc.xNorm;1]; % saving arrays, will be used later
oc.yNormold=[0,oc.yNorm,1];

oc.xNorm=[0;oc.xNorm;1]; % adding 0 and 1 positions
oc.yNorm=[0,oc.yNorm,1]; % adding 0 and 1 positions

[oc.yNorm,oc.xNorm]=meshgrid(oc.yNorm,oc.xNorm);

temp=oc.F_q_2d;
oc.F_q_2d=zeros(size(oc.F_q_2d)+2); % adding the [0,1] points for spanwise positions and
    chordwise positions
oc.F_q_2d(2:end-1,2:end-1)=temp;
oc.F_q_2d(2:end-1,1)=(oc.F_q_2d(2:end-1,3)-oc.F_q_2d(2:end-1,2))./(oc.yNorm(1,3)-oc.yNorm(1,2))
    .* (0-oc.yNorm(1,2))+oc.F_q_2d(2:end-1,2);

figure;
for ii=1:Ns+2
    plot3(oc.xNormold,oc.yNormold(ii)*ones(Nc+2,1),oc.F_q_2d(:,ii),'-','Marker','.');
    hold on;
end
xlabel('x');
ylabel('y');
zlabel('F_q_{2d}');
%axis equal;

```

```

% figure;
% plot3(oc.x,oc.y,oc.F_q, '.');
% xlabel('x');
% ylabel('y');

% % Not necessary, only to check
% oc.Cl_c=sum(oc.F_q,1)./sum(oc.dx,1)./oc.swidth;
% oc.Cl_cref=sum(oc.F_q,1)./oc.swidth/w.cbar;
%
% oc.Cl_c2=sum(oc.dCp.*oc.dx,1)./sum(oc.dx,1);
% oc.Cl_cref2=sum(oc.dCp.*oc.dx,1)./w.cbar
%
%
% figure;
% plot(oc.y,oc.Cl_c,'Marker','.');
% hold on;
% plot(os.y,os.cl_c,'Marker','.');
%
% figure;
% hold on;
% plot(oc.y,oc.Cl_cref,'Marker','.');
% hold on;
% plot(os.y,os.cl_cref,'Marker','.');

end

```

D Code - XFOIL Analysis

This section concerns the scripts used to analyse aerofoils in XFOIL. Due to frequent crashing, freezing, or non-convergence of XFOIL, acquiring Re/α data for each aerofoil must be done manually. After choosing the desired α range, each Re needs to be run individually. Should XFOIL crash, convergence parameters should be modified. Results should be visually inspected to ensure no issues with extrapolation, or too many α values having not converged. The data should then be saved to a .mat file (“lookup table”) containing all of the data. The process should then be repeated until all Re values have been analysed. Optionally, the saved data can be plotted at any point in the analysis.

Function	Description
<code>f01_run1re</code>	Runs one aerofoil, at a specific Re , for a range of α .
<code>f02_save1re</code>	Saves the current Re data to the lookup table.
<code>f03_plotALLre</code>	Plots all the data in the lookup table.

D.1 f01_run1re

```
% Housekeeping
clear;
clc;
close all;

%% Inputs

% Don't forget to ensure each airfoil file includes two (1,0) points and a
% (0,0) point, and is in selig format! (sometimes you need thick trailing
% edge, so use (1,0.001) points))

afname='Mh93.dat';

alphaBounds=[-10,20]; % make sure these are consistent with every foil?

M=0; %mach number
% REVALS - WHAT YOU SHOULD RUN IT AT (you can slightly change, just a
% recommendation: (in millions) 1 2 5 10 20 50 100 200 500 800
Re= 93 *1000 *1000; % MANUALLY SET REYNOLDS NUMBER WITHIN YOUR RANGE

% XFOIL CONVERGENCE PARAMETERS
operIter=300; %%%%%%%%%%%%% CHANGE MAX ITERATIONS UNTIL CONVERGENCE
nPanels =5000; %%%%%%%%%%%%% CHANGE NUMBER OF PANELS

% Max AoA for which code looks for clmax, in degrees
maoa=50; % if last cl value is lower than clmax, can set to whatever large value u want (eg.
50)

% Smooth span
grad=5; % MUST BE AN ODD NUMBER

%% Computations and running xfoil

if mod(grad,2)==0 % if grad is even
    error('MAKE GRAD ODD');
end

% Airfoil coordinates
fid=fopen(afname);
afCoords=fscanf(fid,'%f',[2,inf]);
fclose(fid);
```

```

% plot(afCoords(:,1),afCoords(:,2),'-k','Marker','.')
% axis equal
% return

% XFOIL parameters
da=0.5;
alphaPos=[0:da:alphaBounds(2)]; % positive AoAs. Xfoil runs better if you run from 0 to
    alphamax, then 0 to alphamin for some reason
alphaNeg=[-da:-da:alphaBounds(1)]; %negative AoAs
alphaFull=[flip(alphaNeg),alphaPos]; % full range of AoAs, for interpolating later

% Running XFOIL
% [pol,uhh]=xfoil(afCoords,00,Re,M,['oper iter ',num2str(operIter)],['panels n ',num2str(
    nPanels)]);
% plot(uhh.xcp,-uhh.cp)
% min(uhh.cp)
% return;
[poll,~]=xfoil(afCoords,alphaNeg,Re,M,['oper iter ',num2str(operIter)],['panels n ',num2str(
    nPanels)]); % runs xfoil at negative AOAs
[pol2,~]=xfoil(afCoords,alphaPos,Re,M,['oper iter ',num2str(operIter)],['panels n ',num2str(
    nPanels)]); % at positive AoAs

% Reformats xfoil outputs
alpha=[flip(poll.alpha);pol2.alpha]; %the AoAs that converged
clorig=[flip(poll.CL);pol2.CL]; % the aero coeffs that converged
cdorig=[flip(poll.CD);pol2.CD];
cmorig=[flip(poll.Cm);pol2.Cm];
clcdorig=clorig./cdorig;

% cl=smooth(clorig);
% cd=smooth(cdorig);
% cm=smooth(cmorig);
% clcd=smooth(clcdorig);
cl=clorig;
cd=cdorig;
cm=cmorig;
clcd=clcdorig;

Clhere=interp1(alpha,cl,alphaFull,'pchip'); % interpolates for every point, even the ones
    xfoil didnt converge at
Cdhere=interp1(alpha,cd,alphaFull,'pchip');
Cmhere=interp1(alpha,cm,alphaFull,'pchip');
ClCdhere=interp1(alpha,clcd,alphaFull,'pchip');

Clhere =smooth(Clhere,grad); % smooths the curves to remove some of the noise
Cdhere =smooth(Cdhere,grad);
Cmhere =smooth(Cmhere,grad);
ClCdhere=smooth(ClCdhere,grad);
clear('grad');

% Findin max cl - sometimes the var right of the aoa range is larger than
% clmax. this gives us a wrong value of cl
if maoa<alphaFull(end)
    clmi=find(alphaFull<=maoa);
else
    clmi=length(alphaFull);
end
clmi=max(clmi); % index in cl array corresponding to max AoA to search for clmax

[mmm,iii]=max(Clhere(1:clmi)); % Finds max cl
Clmaxhere=[alphaFull(iii),Re,mmm]; % saves clmax, alpha and Re at which occurs

```

```

[mmmm,iiii]=max(ClCdhere); % Same but for max cl/cd
ClCdmaxhere=[alphaFull(iiii),Re,mmmm];

if 1 % TO PLOT

    % NOTE: the crosses are the points xfoil converged at. The lines are
    % the points that we interpolated at, from the xfoil data.

    wx=700;
    wy=650;
    figure('Position',[ (1920-wx)/2,100,wx,wy]); %cl curve. Shows converged points, and
        interpolated points

    subplot(4,1,1)
    plot(alpha,clorig,'x');
    hold on;
    plot(alphaFull,Clhere,'-');
    plot(Clmaxhere(1),Clmaxhere(3),'ko','MarkerSize',8);
    title('cl vs alpha');
    xlabel('alpha');
    ylabel('cl');

    subplot(4,1,2) %cd curve
    plot(alpha,cdorig,'x');
    hold on;
    plot(alphaFull,Cdhere,'-');
    title('cd vs alpha');
    xlabel('alpha');
    ylabel('cd');

    subplot(4,1,3) %cm curve
    plot(alpha,cmorig,'x');
    hold on;
    plot(alphaFull,Cmhere,'-');
    title('cm vs alpha');
    xlabel('alpha');
    ylabel('cm');

    subplot(4,1,4) %cd/cd curve
    plot(alpha,clcdorig,'x');
    hold on;
    plot(alphaFull,ClCdhere,'-');
    plot(ClCdmaxhere(1),ClCdmaxhere(3),'ko','MarkerSize',8);
    title('cl/cd vs alpha');
    xlabel('alpha');
    ylabel('cl/cd');

    sgtitle(['Re = ',num2str(Re/1000000),' million.']);
end

%% Saving
pf.a=alphaFull';
pf.cl=Clhere;
pf.cd=Cdhere;
pf.cm=Cmhere;
pf.clcd=ClCdhere;
pf.clmax=Clmaxhere;
pf.clcdmax=ClCdmaxhere;
save('here','pf','afname','Re');

```

D.2 f02_save1re


```

% XFOILSAVE - This saves the last iteration data from main.m into a .mat
% file
clear;
clc;
close all;

load('here.mat');

fname=[afname(1:end-4), '.mat'];

if isfile(fname) % if the file already exists, add a new column for data
    % Opens the files
    load(fname) % .mat file for all data

    [~,m]=size(thisfoil.re);
    ii=m+1; % number of reynolds numbers already elapsed +1 - adds a new row
else % if the file does not exist, start from the first row
    ii=1; % first row for first dataset
end

% Adds the data as new columns to the file
thisfoil.a(:,ii)=pf.a;
thisfoil.cl(:,ii)=pf.cl;
thisfoil.cd(:,ii)=pf.cd;
thisfoil.cm(:,ii)=pf.cm;
thisfoil.clcd(:,ii)=pf.clcd;
thisfoil.clmax(:,ii)=pf.clmax;
thisfoil.clcdmax(:,ii)=pf.clcdmax;
thisfoil.re(:,ii)=Re*ones(length(pf.a),1);

% saves the file
save(fname, 'thisfoil');

```

D.3 f03_plotALLre

```

% Plots everything
clear;
clc;
close all;

% Boolean
dimPlot=2; % 2 for a 2d plot with multiple curves plotted, 3 for a full 3d plot

load('here.mat'); % loads the last file created by f01.run1re
load(afname(1:end-4)); % loads the multiple-reynolds file

% Get legend entries
for ii=1:size(thisfoil.re,2) % for every column
    lname{ii}=num2str(thisfoil.re(1,ii)/10^6);
end

figure;

subplot(2,2,1); % C1 plot
plot3(thisfoil.a,thisfoil.re/1000000,thisfoil.cl, '-'); % the ' symbols are so that each line is
    C1 vs alpha, not cl vs Re
hold on;
plot3(thisfoil.clmax(1,:),thisfoil.clmax(2,+)/1000000,thisfoil.clmax(3,:), 'kx'); % plots clmax
    points
xlabel('alpha');
ylabel('Re (million)');

```

```

xlabel('Cl');
title('Cl contour');

if dimPlot==2
    view([0,0]); % Delete this and line below if needed
    legend(lname)
end

subplot(2,2,2); % Cd plot
plot3(thisfoil.a,thisfoil.re/1000000,thisfoil.cd,'-');
xlabel('alpha');
ylabel('Re (million)');
zlabel('Cd');
title('Cd contour');

if dimPlot==2
    view([0,0]);
    legend(lname)
end

xlim([-5,10]);

subplot(2,2,3); % Cm plot
plot3(thisfoil.a,thisfoil.re/1000000,thisfoil.cm,'-');
xlabel('alpha');
ylabel('Re (million)');
zlabel('Cm');
title('Cm contour');

if dimPlot==2
    view([0,0]); % Delete this and line below if needed
    legend(lname)
end

subplot(2,2,4); % Cl/Cd plot
plot3(thisfoil.a,thisfoil.re/1000000,thisfoil.clcd,'-'); % the ' symbols are so that each line
    is Cl vs alpha, not cl vs Re
hold on;
plot3(thisfoil.clcdmax(1,:),thisfoil.clcdmax(2,:)/1000000,thisfoil.clcdmax(3,:), 'kx'); % plots
    clmax points
xlabel('alpha');
ylabel('Re (million)');
zlabel('Cl/Cd');
title('Cl/Cd contour');

if dimPlot==2
    view([0,0]); % Delete this and line below if needed
    legend(lname)
end

```

E Code - Common Helper Scripts

This section concerns the scripts common to both the conventional and BWB codes, that are used for very specific purposes.

Function	Description
<code>myAvlFileRead</code>	Looks for specific values in AVL output files.
<code>spacing</code>	Calculates panel spacings.
<code>structures_chordwise_interp</code>	Interpolates and outputs data for structures.

E.1 `myAvlFileRead`

```
function[out]=myAvlFileRead(fid,namefind)
% MYAVLFILEREAD - scans through fid until it finds a specific string. Then
% outputs the value corresponding to that string.
%
% i.e. if the file has the following line:
% Alpha = 2.852
%
% this function will look for "Alpha" and output the value after the =
% sign.
%
% INPUTS:
%     fid - file identifier for currently open file.
%     namefind - string to look for
%
% OUPUTS:
%     out - value to be outputted.

while 1
    %pause(0.05);
    temp=fscanf(fid,'%s',1); % reads the next string
    if length(temp)==length(namefind) % checks if the strings have the same length
        if temp==namefind % checks if the strings match
            fscanf(fid,'%s,1'); % reads the = sign
            out=str2num(fscanf(fid,'%s',1)); % reads the value.
            break; % Stops
        end
    end
end
end
end
```

E.2 `spacing`

```
function[out]=spacing(N,S)
% SPACING - generates spacing according to AVL conventions:
% Cheat sheet for Cspace and Sspace
%     parameter                spacing
%     -----
%
%     3.0      equal           |  |  |  |  |  |  |  |  |
%
%     2.0      sine            || |  |  |  |  |  |  |
%
%     1.0      cosine          || |  |  |  |  |  |  ||
%
%     0.0      equal           |  |  |  |  |  |  |  |  |
```

```

%   -1.0      cosine      || |   |   |   |   |   |
%
%   -2.0      -sine       |   |   |   |   |   |
%
%   -3.0      equal       |   |   |   |   |   |
%
% An intermediate parameter value will result in a blended distribution.
%
% INPUT: N (# panels), S (spacing parameter); OUTPUT: distribution from 0 to 1

% Different spacings
de=0:1/N:1; % equal
dc=1/2-cos(de*pi)/2; % cosine
ds=1-cos(de*pi/2); % sin
dns=flip(cos(de*pi/2)); % neg sin

% Exact distributions
if S==3 || S==0 || S==-3 % equal
    out=ed;
    return;
elseif S==1 || S==-1 % cosine
    out=dc;
    return;
elseif S==2 % sin
    out=ds;
    return;
elseif S==-2 % neg sin
    out=dns;
    return;
end

% Blended distributions
d=[de;ds;dc;de;dc;dns;de]; % array of distributions
v=[-3,-2,-1,0,1,2,3]; % values corresponding
[~,ii]=find(v>S,1); % finds the first value that is greater than the input
out=(v(ii)-S)*d(ii-1,:)+(S-v(ii-1))*d(ii,:); % blends between 2 distributions

end

```

E.3 structures_chordwise_interp

```

% STRUCTURES_CHORDWISE_INTERP
% Must be run after either conv.main or bw.b.main
% Interpolates a higher-quality mesh for the distribution of loads along
% the span and chord of the wing.

% Housekeeping
close all;

% Inputs that structures would want
str.ss=0:0.005:1; % Spanwise stations. Note you'll get forces for the "strips" between these
    stations. Note also that a spanwise station is just y/s, or y/(b/2).
str.cs=0:0.01:1; % Chordwise stations.

loads=chordwiseInterp(oc,str,w); % this is force / q = force / (0.5 rho V^2)

%% Outputting as they like
str.ss=str.ss';
normalised.x=loads.Cnorm'; % normalised data (x and y normalised from 0 to 1)
normalised.y=loads.Snorm;

```

```

normalised.F=loads.F;

absolute.y=w.s*loads.Snrm; % absolute data (x and y varying in the shape of the planform)
absolute.x=interp1(w.ss,w.xLE,loads.Snrm,'linear')+loads.Cnorm'.*interp1(w.ss,w.c,loads.Snrm,
    'linear');
absolute.F=loads.F;

save('bwb_structures_out','normalised','absolute');

figure; % plots normalised data
for ii=1:length(normalised.y)
    %plot3(loads.Cnorm,loads.Snrm(ii)*ones(size(str.cs-1)),loads(:,ii));
    plot3(normalised.x,normalised.y(ii)*ones(size(normalised.x)),normalised.F(:,ii),'-','Marker','.'); % plots normalised span and chord, and load.
    hold on;
end

xlabel('x (%chord)');
ylabel('y (%chord)');
zlabel('F-q');

figure; % plots absolute data
for ii=1:length(absolute.y)
    %plot3(loads.Cnorm,loads.Snrm(ii)*ones(size(str.cs-1)),loads(:,ii));
    plot3(absolute.x(:,ii),absolute.y(ii)*ones(size(absolute.x)),absolute.F(:,ii),'-','Marker','.'); % plots normalised span and chord, and load.
    hold on;
end

xlabel('x (abs)');
ylabel('y (abs)');
zlabel('F-q');
%axis equal

function[out]=chordwiseInterp(oc,str,w)
posy=(str.ss(2:end)+str.ss(1:end-1))/2; % normalised y-coordinates, queried
posx=(str.cs(2:end)+str.cs(1:end-1))/2; % normalised x-coordinates, queried

[Y,X]=meshgrid(posy,posx); % meshgrid

F_q_2di=interp2(oc.yNorm,oc.xNorm,oc.F_q_2d,Y,X,'linear'); % interpolates the given data

out.F=F_q_2di.*(str.ss(2:end)-str.ss(1:end-1))*w.s; % calculates actual force
out.Cnorm=posx;
out.Snrm=posy;
end

```